

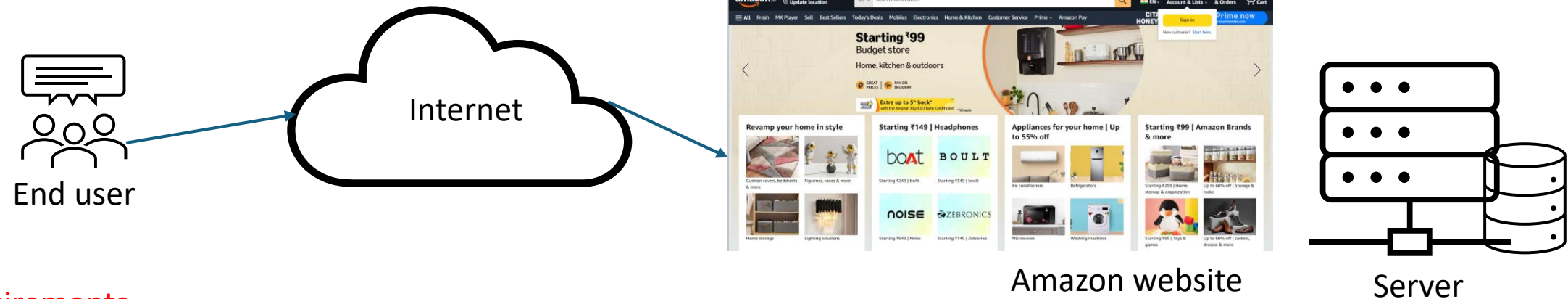
Secure Socket Layer

Module VII

Why TLS?

- Transport layer security provides end-to-end security services for applications that use a reliable transport layer protocol such as TCP.
- The idea is to provide security services for transactions on the Internet.

when a customer shops online



Requirements

The customer needs to be sure that the server belongs to the actual vendor, not an impostor

The customer and the vendor need to be sure that the contents of the message are not modified during transmission

The customer and the vendor need to be sure that an impostor does not intercept sensitive information such as a credit card number

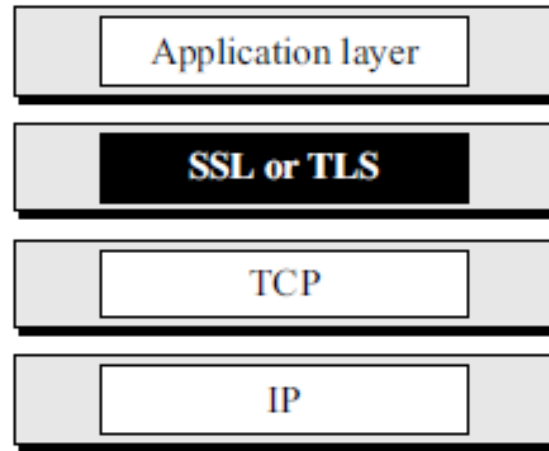
Entity authentication

Message Integrity

Confidentiality

SSL/TLS

- Two protocols are dominant today for providing security at the transport layer: the Secure Sockets Layer (SSL) Protocol and the Transport Layer Security (TLS) Protocol.



Position of SSL/TLS in the Internet protocol model

- One of the goals of these protocols is to provide server and client authentication, data confidentiality, and data integrity. Application-layer client/server programs, such as Hypertext Transfer Protocol (HTTP), that use the services of TCP can encapsulate their data in SSL packets.
- If the server and client are capable of running SSL (or TLS) can run, then the client can use the URL `https://...` instead of `http://...` to allow HTTP messages to be encapsulated in SSL (or TLS) packets.

A website that implements SSL/TLS has "HTTPS" in its URL instead of "HTTP."

HTTP vs HTTPS



SSL Architecture

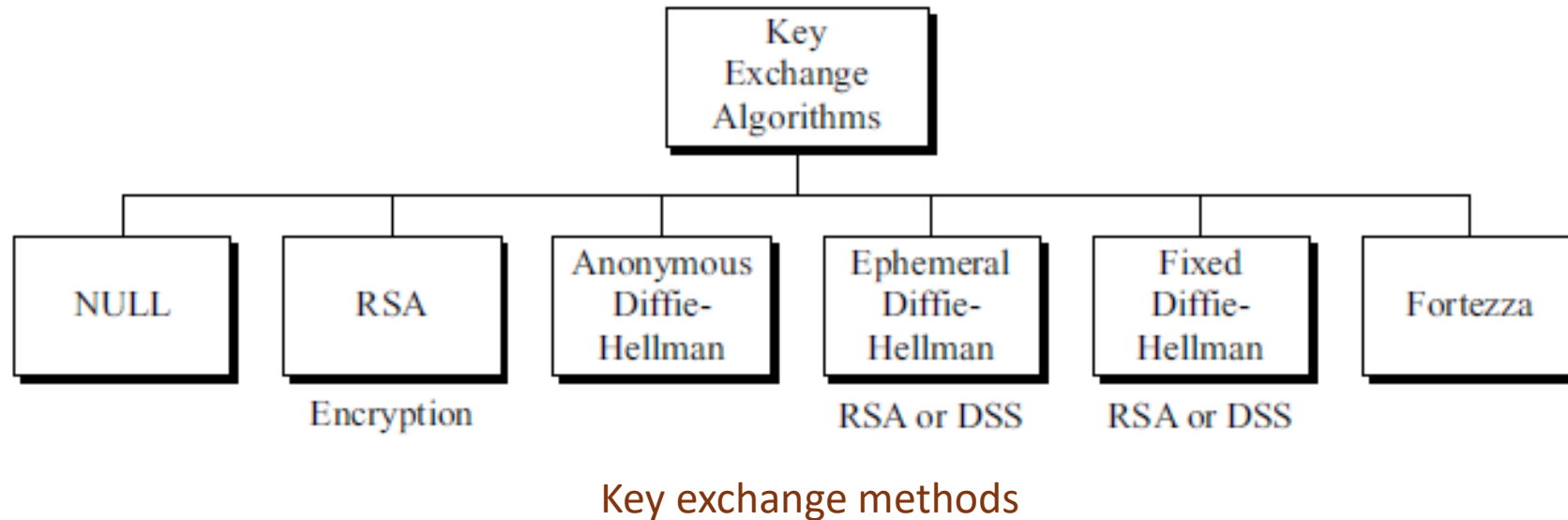
- SSL is designed to provide security and compression services to data generated from the application layer. SSL can receive data from any application layer protocol, but usually, the protocol is HTTP.
- Netscape developed SSL in 1994. Versions 2 and 3 were released in 1995. The present discussion is on SSLv3.
- The data received from the application is compressed (optional), signed, and encrypted. The data is then passed to a reliable transport layer protocol such as TCP.

SSL services

- SSL provides several services on data received from the application layer: Fragmentation, Compression, Message Integrity and Confidentiality.
- In **fragmentation**, first, SSL divides the data into blocks of 214 bytes or less.
- Each fragment of data is **compressed** using one of the lossless compression methods negotiated between the client and server. This service is optional.
- To preserve the **integrity of data**, SSL uses a keyed-hash function to create a MAC.
- To provide **confidentiality**, the original data and the MAC are encrypted using symmetric key cryptography.
- A header is added to the encrypted payload. The payload is then passed to a reliable transport layer protocol (**framing**).

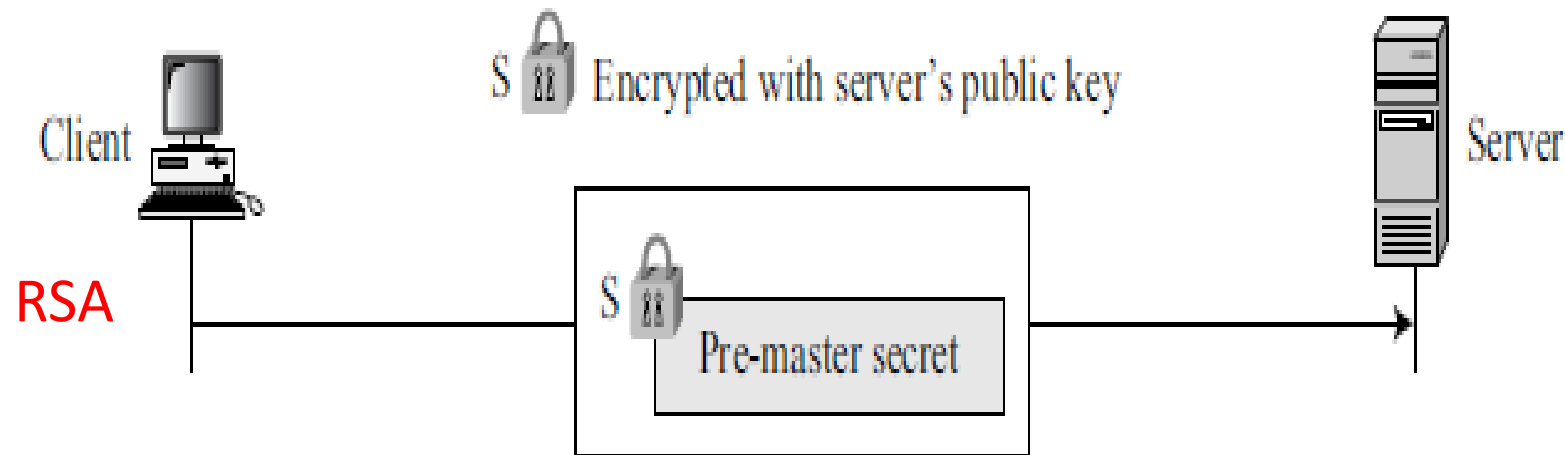
Key Exchange Algorithms

- To exchange an authenticated and confidential message, the client and the server each need six cryptographic secrets (four keys and two initialization vectors).
- However, to create these secrets, one pre-master secret must be established between the two parties. SSL defines six key-exchange methods to establish this premaster secret: NULL, RSA, anonymous Diffie-Hellman, ephemeral Diffie-Hellman, fixed Diffie-Hellman, and Fortezza.



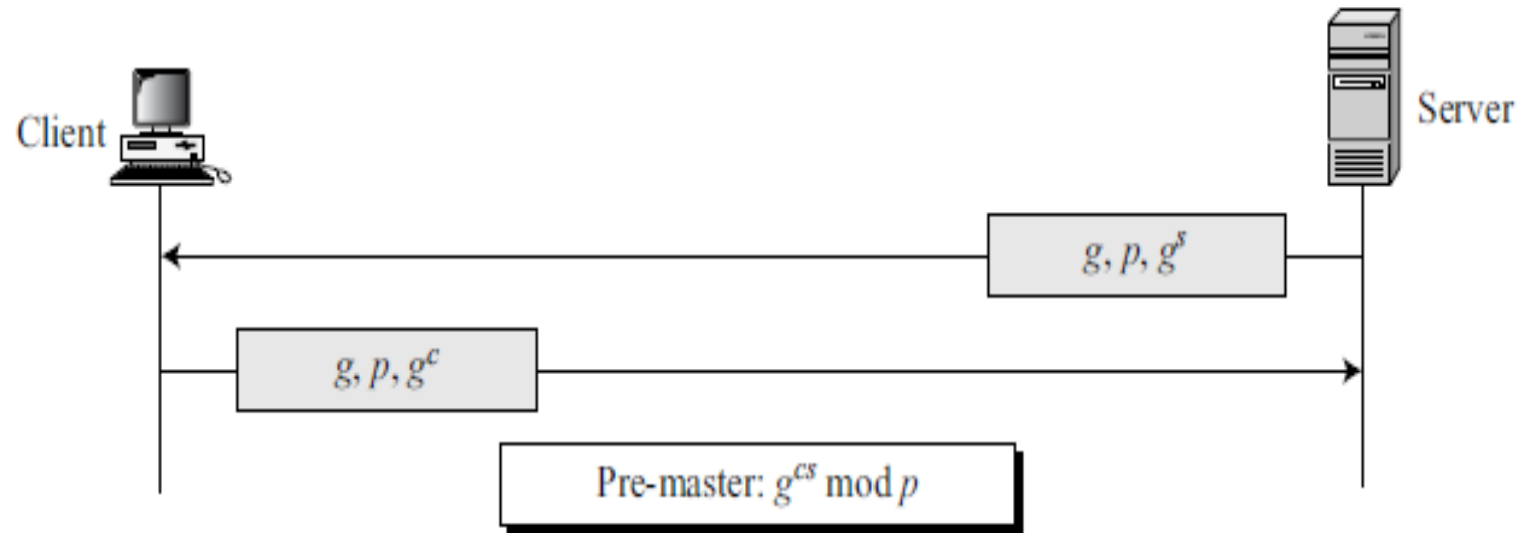
Key Exchange Methods

- **NULL**, there is no key exchange in this method. No pre-master key is established between the client and the server. Both client and server need to know the value of the pre-master secret.
- **RSA**, the pre-master secret is a 48-byte random number created by the client, encrypted with the server's RSA public key and sent to the server. The server needs to send its RSA encryption/decryption certificate.



Key Exchange Methods

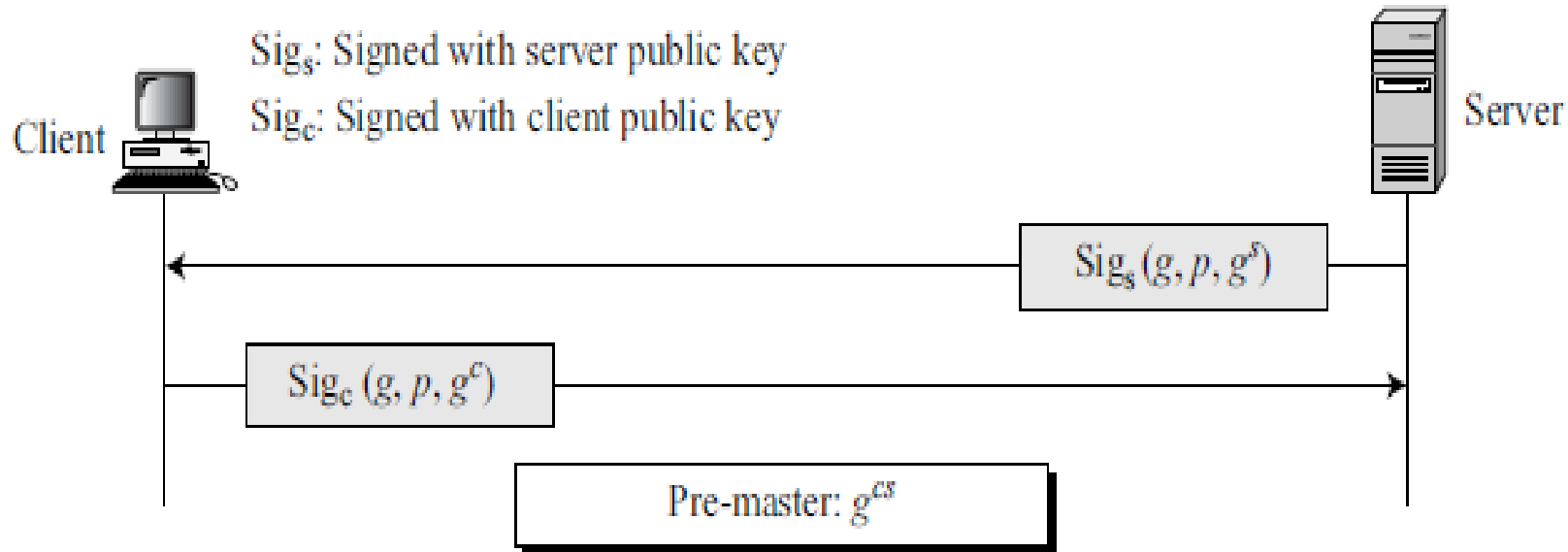
- **Anonymous Diffie-Hellman** is the simplest and most insecure method. The pre-master secret is established between the client and server using the Diffie-Hellman (DH) protocol. The Diffie-Hellman half-keys are sent in plaintext. It is called anonymous Diffie-Hellman because neither party is known to the other. The most serious disadvantage of this method is the **man-in-the-middle** attack.



Anonymous Diffie-Hellman Key exchange

Key Exchange Methods

- **Ephemeral Diffie-Hellman:** To thwart the man-in-the-middle attack, the ephemeral Diffie-Hellman key exchange can be used. Each party sends a Diffie-Hellman key signed by its private key. The receiving party needs to verify the signature using the public key of the sender. The public keys for verification are exchanged using either RSA or DSS digital signature certificates.



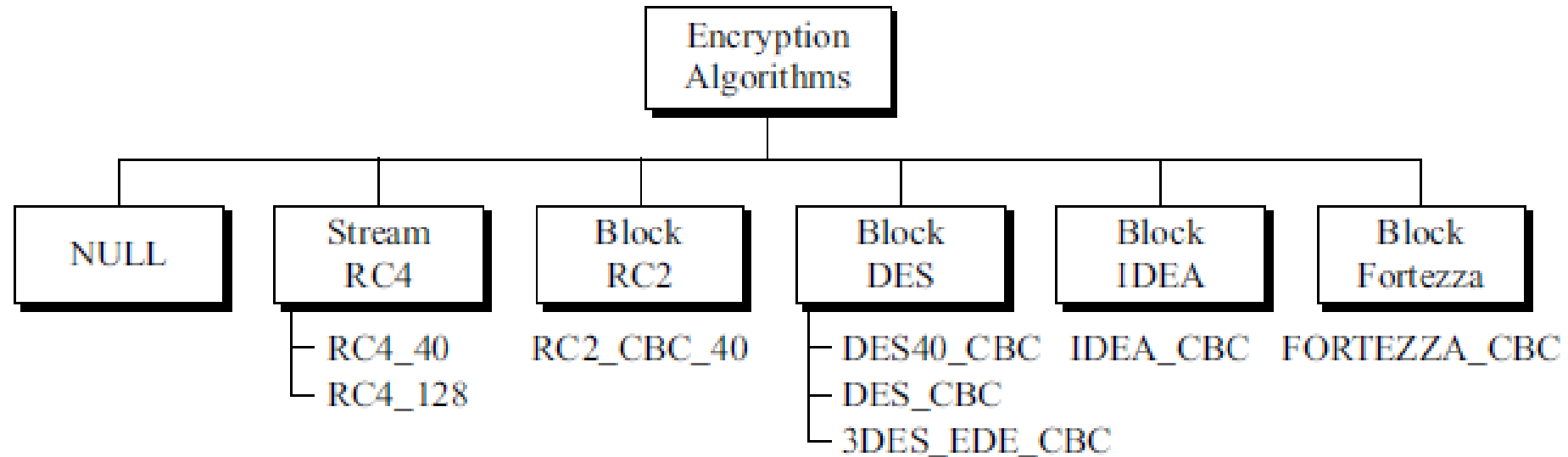
Ephemeral Diffie-Hellman Key exchange

Key Exchange Methods: Fixed Diffie-Hellman

- Another solution is the fixed Diffie-Hellman method. All entities in a group can prepare fixed Diffie-Hellman parameters (g and p).
- Then each entity can create a fixed Diffie-Hellman half-key (g^x). For additional security, each individual half-key is inserted into a certificate verified by a certification authority (CA). In other words, the two parties do not directly exchange the half-keys; the CA sends the half-keys in an RSA or DSS special certificate.
- When the client needs to calculate the pre-master, it uses its own fixed half-key and the server half-key received in a certificate.
- The server does the same, but in the reverse order. Note that no key-exchange messages are passed in this method; only certificates are exchanged.

Encryption/Decryption Algorithms

- There are several choices for the encryption/decryption algorithm. There are six groups of encryption/decryption algorithms as per literature. All block protocols use an 8-byte initialization vector (IV) except for Fortezza, which uses a 20-byte IV.

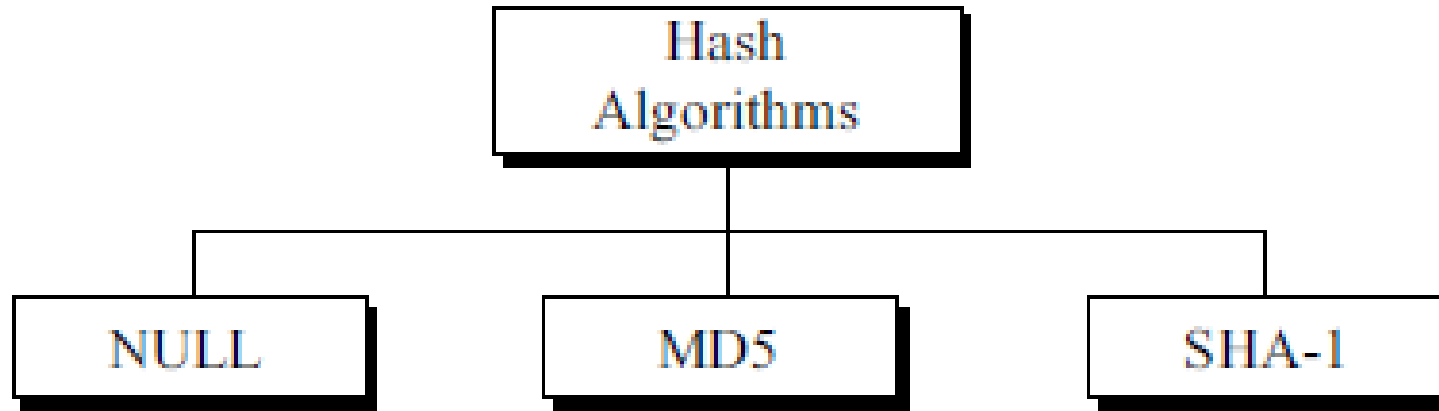


Encryption/Decryption Algorithms

- The **NULL** category simply defines the lack of an encryption/decryption algorithm.
- **Stream RC**: Two RC algorithms are defined in stream mode: RC4-40 (40-bit key) and RC4-128 (128-bit key).
- **Block RC**: One RC algorithm is defined in block mode: RC2_CBC_40 (40-bit key).
- **DES**: All DES algorithms are defined in block mode. DES40_CBC uses a 40-bit key. Standard DES is defined as DES_CBC. 3DES_EDE_CBC uses a 168-bit key.
- **IDEA**: The one IDEA algorithm defined in block mode is IDEA_CBC, with a 128-bit key.
- **Fortezza**: The one Fortezza algorithm defined in block mode is FORTEZZA_CBC, with a 96-bit key.

Hash Algorithms

- SSL uses hash algorithms to provide message integrity (message authentication). Three hash functions are defined: NULL, MD5 and SHA-1.



- **Null**: The two parties may decline to use an algorithm. In this case, there is no hash function, and the message is not authenticated.
- **MD5**: The two parties may choose MD5 as the hash algorithm. In this case, a 128-key MD5 hash algorithm is used.
- **SHA-1**: The two parties may choose SHA as the hash algorithm. In this case, a 160-bit SHA-1 hash algorithm is used.

Cipher suite

The combination of key exchange, hash, and encryption algorithms defines a cipher suite for each SSL session.

Each suite starts with the term “SSL” followed by the key exchange algorithm. The word “WITH” separates the key exchange algorithm from the encryption and hash algorithms.

SSL_DHE_RSA_WITH_DES_CBC_SHA

For example, the above cipher suite defines DHE_RSA (ephemeral Diffie-Hellman with RSA digital signature) as the key exchange with DES_CBC as the encryption algorithm and SHA as the hash algorithm.

<i>Cipher suite</i>	<i>Key Exchange</i>	<i>Encryption</i>	<i>Hash</i>
SSL_NULL_WITH_NULL_NULL	NULL	NULL	NULL
SSL_RSA_WITH_NULL_MD5	RSA	NULL	MD5
SSL_RSA_WITH_NULL_SHA	RSA	NULL	SHA-1
SSL_RSA_WITH_RC4_128_MD5	RSA	RC4	MD5
SSL_RSA_WITH_RC4_128_SHA	RSA	RC4	SHA-1
SSL_RSA_WITH_IDEA_CBC_SHA	RSA	IDEA	SHA-1
SSL_RSA_WITH_DES_CBC_SHA	RSA	DES	SHA-1
SSL_RSA_WITH_3DES_EDE_CBC_SHA	RSA	3DES	SHA-1
SSL_DH_anon_WITH_RC4_128_MD5	DH_anon	RC4	MD5
SSL_DH_anon_WITH_DES_CBC_SHA	DH_anon	DES	SHA-1
SSL_DH_anon_WITH_3DES_EDE_CBC_SHA	DH_anon	3DES	SHA-1
SSL_DHE_RSA_WITH_DES_CBC_SHA	DHE_RSA	DES	SHA-1
SSL_DHE_RSA_WITH_3DES_EDE_CBC_SHA	DHE_RSA	3DES	SHA-1
SSL_DHE_DSS_WITH_DES_CBC_SHA	DHE_DSS	DES	SHA-1
SSL_DHE_DSS_WITH_3DES_EDE_CBC_SHA	DHE_DSS	3DES	SHA-1
SSL_DH_RSA_WITH_DES_CBC_SHA	DH_RSA	DES	SHA-1
SSL_DH_RSA_WITH_3DES_EDE_CBC_SHA	DH_RSA	3DES	SHA-1
SSL_DH_DSS_WITH_DES_CBC_SHA	DH_DSS	DES	SHA-1
SSL_DH_DSS_WITH_3DES_EDE_CBC_SHA	DH_DSS	3DES	SHA-1
SSL_FORTEZZA_DMS_WITH_NULL_SHA	Fortezza	NULL	SHA-1
SSL_FORTEZZA_DMS_WITH_FORTEZZA_CBC_SHA	Fortezza	Fortezza	SHA-1
SSL_FORTEZZA_DMS_WITH_RC4_128_SHA	Fortezza	RC4	SHA-1

Compression algorithms

- Compression is optional in SSLv3. No specific compression algorithm is defined for SSLv3. Therefore, the default compression method is NULL. However, a system can use whatever compression algorithm it desires.

Cryptographic parameter generation

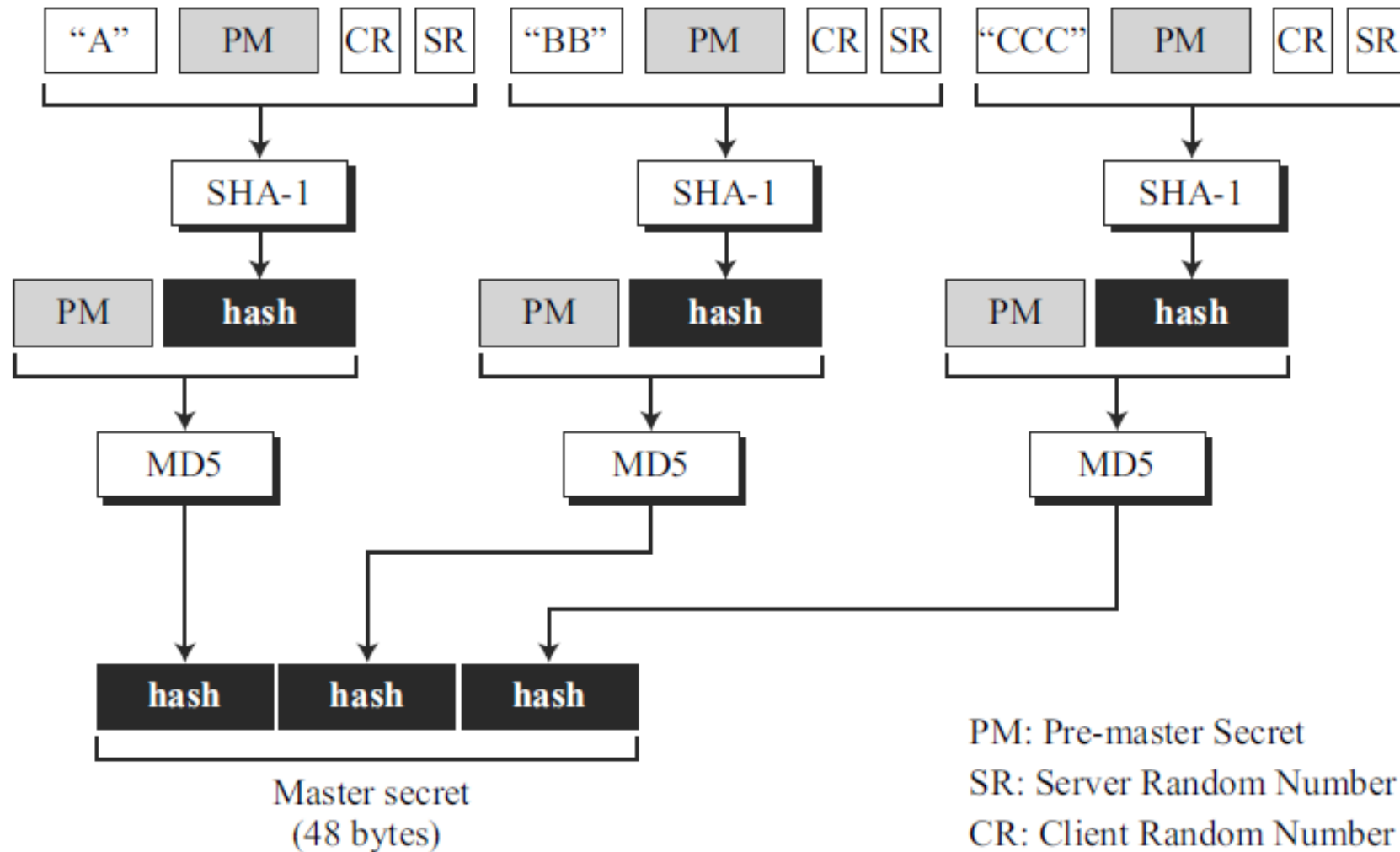
- To achieve message integrity and confidentiality, SSL needs six cryptographic secrets, four keys and two IVs.
- The client needs one key for message authentication (HMAC), one key for encryption, and one IV for block encryption. The server needs the same.
- SSL requires that the keys for one direction be different from those for the other direction. This is because, If there is an attack in one direction, the other direction is not affected.

The parameters are generated using the following procedure

- The client and server exchange two random numbers; one is created by the client and the other by the server.
- The client and server exchange one pre-master secret using one of the key exchange algorithms (discussed in previous slides).

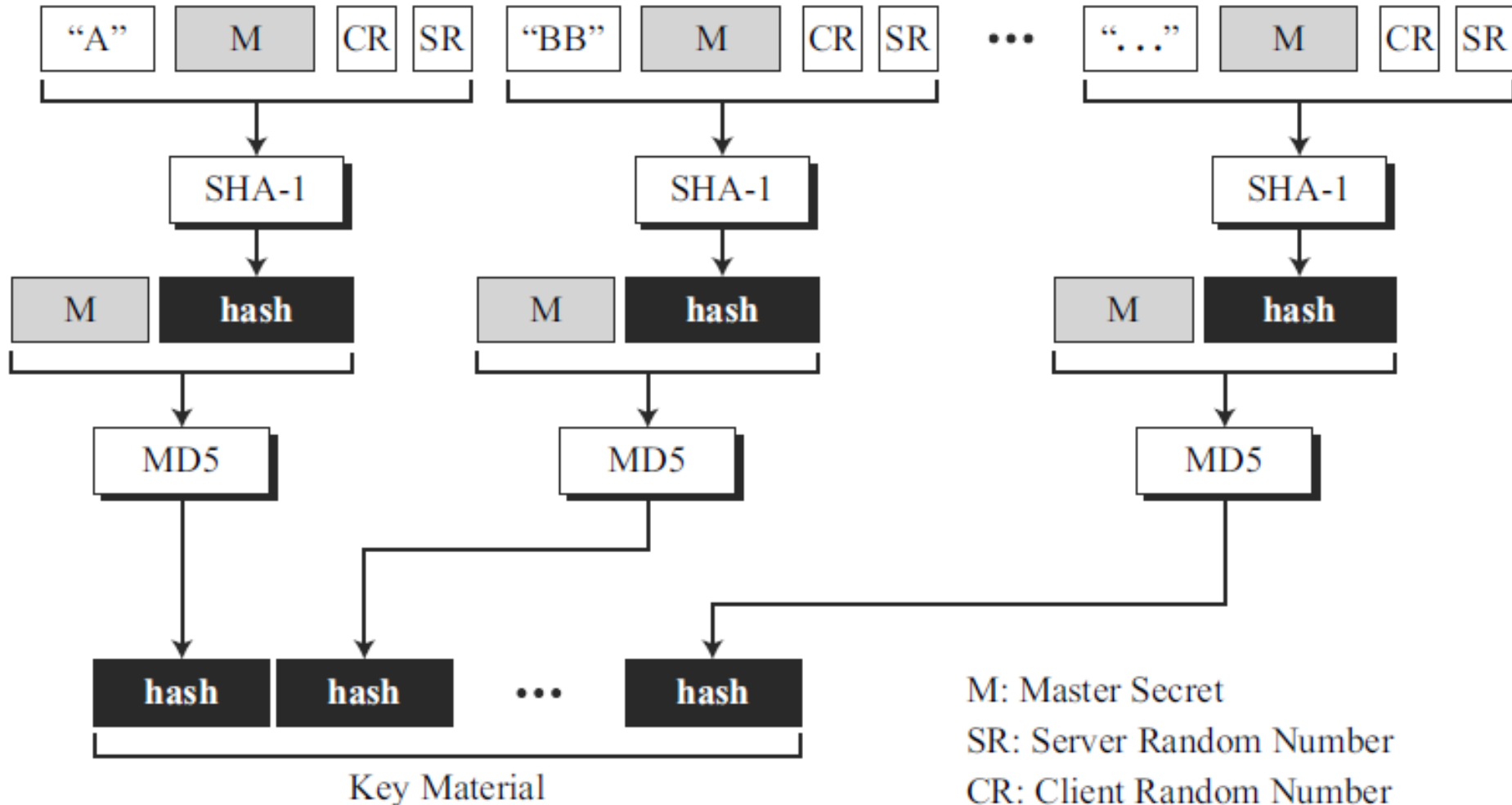
The parameters are generated using the following procedure (cntd..)

- A 48-byte master secret is created from the pre-master secret by applying two hash functions (SHA-1 and MD5).



The parameters are generated using the following procedure (cntd..)

- The master secret is used to create variable-length key material by applying the same set of hash functions and prepending with different constants



The length of the key material block depends on the cipher suite selected and the size of keys needed for this suite.

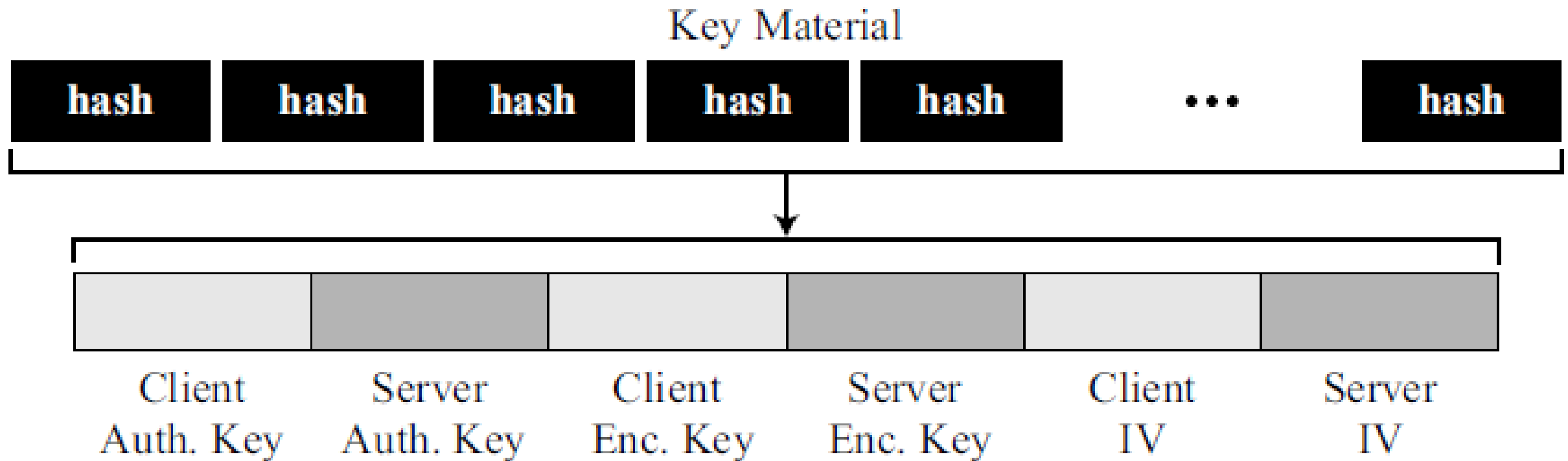
The parameters are generated using the following procedure (cntd..)

- Six different keys are extracted from the key material.

Auth. Key: Authentication Key

Enc. Key: Encryption Key

IV: Initialization Vector

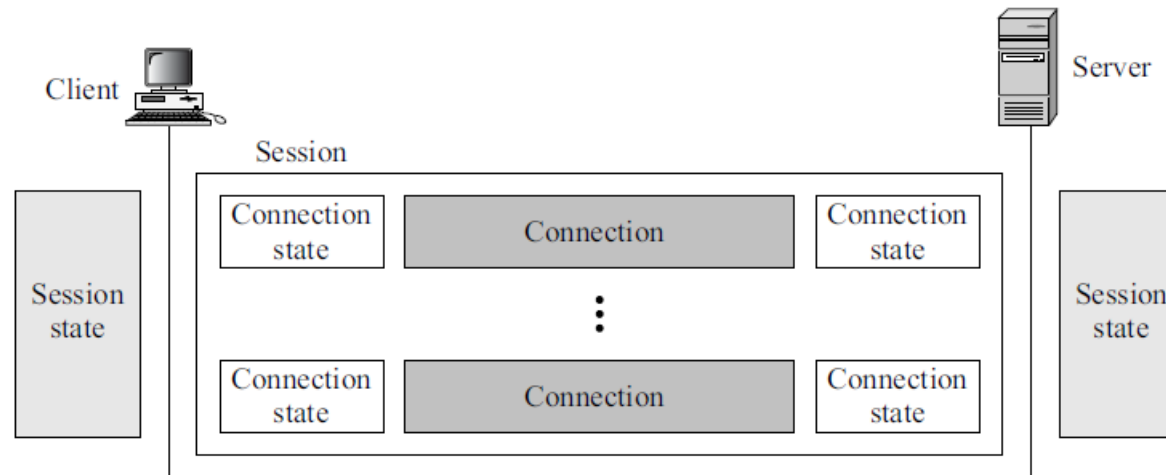


Sessions and connections

- SSL differentiates a connection from a session.
- A session is an association between a client and a server.
- After a session is established, the two parties have common information such as the session identifier, the certificate authenticating each of them (if necessary), the compression method (if needed), the cipher suite, and a master secret that is used to create keys for message authentication encryption.
- For two entities to exchange data, the establishment of a session is necessary, but not sufficient; they need to create a connection between themselves. The two entities exchange two random numbers and create, using the master secret, the keys and parameters needed for exchanging messages involving authentication and privacy.
- A session can consist of many connections. A connection between two parties can be terminated and reestablished within the same session. When a connection is terminated, the two parties can also terminate the session, but it is not mandatory.
- A session can be suspended and resumed again.

Sessions and connections –cntd..

- To create a new session, the two parties need to go through a negotiation process.
- To resume an old session and create only a new connection, the two parties can skip part of the negotiation process and go through a shorter one.
- There is no need to create a master secret when a session is resumed.
- The separation of a session from a connection prevents the high cost of creating a master secret. By allowing a session to be suspended and resumed, the process of the master secret calculation can be eliminated.



In a session, one party has the role of a client and the other the role of a server; In a connection, both parties have equal roles, they are peers.

Session state

- A session is defined by a session state, a set of parameters established between the server and the client.

<i>Parameter</i>	<i>Description</i>
Session ID	A server-chosen 8-bit number defining a session.
Peer Certificate	A certificate of type X509.v3. This parameter may be empty (null).
Compression Method	The compression method.
Cipher Suite	The agreed-upon cipher suite.
Master Secret	The 48-byte secret.
Is resumable	A yes-no flag that allows new connections in an old session.

list of parameters for a session state

Connection state

- A connection is defined by a connection state, a set of parameters established between two peers.

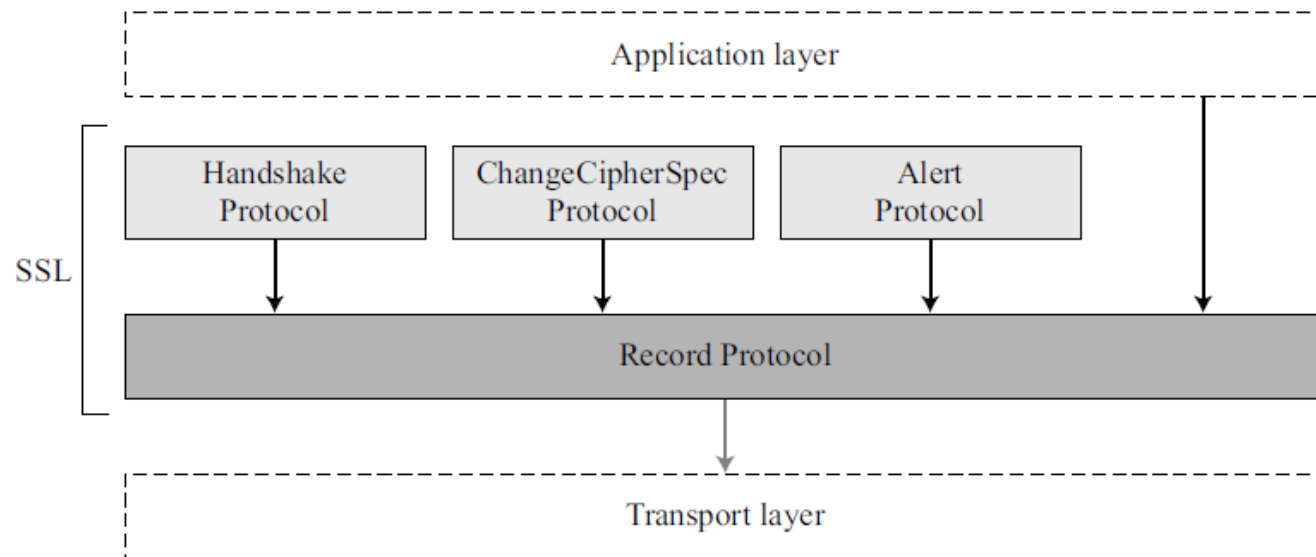
<i>Parameter</i>	<i>Description</i>
Server and client random numbers	A sequence of bytes chosen by the server and client for each connection.
Server write MAC secret	The outbound server MAC key for message integrity. The server uses it to sign; the client uses it to verify.
Client write MAC secret	The outbound client MAC key for message integrity. The client uses it to sign; the server uses it to verify.
Server write secret	The outbound server encryption key for message integrity.
Client write secret	The outbound client encryption key for message integrity.
Initialization vectors	The block ciphers in CBC mode use initialization vectors (IVs). One initialization vector is defined for each cipher key during the negotiation, which is used for the first block exchange. The final cipher text from a block is used as the IV for the next block.
Sequence numbers	Each party has a sequence number. The sequence number starts from 0 and increments. It must not exceed $2^{64} - 1$.

Connection state parameters

Connection state (cntd..)

- SSL uses two attributes to distinguish cryptographic secrets: write and read.
- The term write specifies the key used for signing or encrypting outbound messages.
- The term read specifies the key used for verifying or decrypting inbound messages.
- The write key of the client is the same as the read key of the server; the read key of the client is the same as the write key of the server.
- The client and the server have six different cryptography secrets: three read secrets and three write secrets. The read secrets for the client are the same as the write secrets for the server and vice versa.

FOUR PROTOCOLS

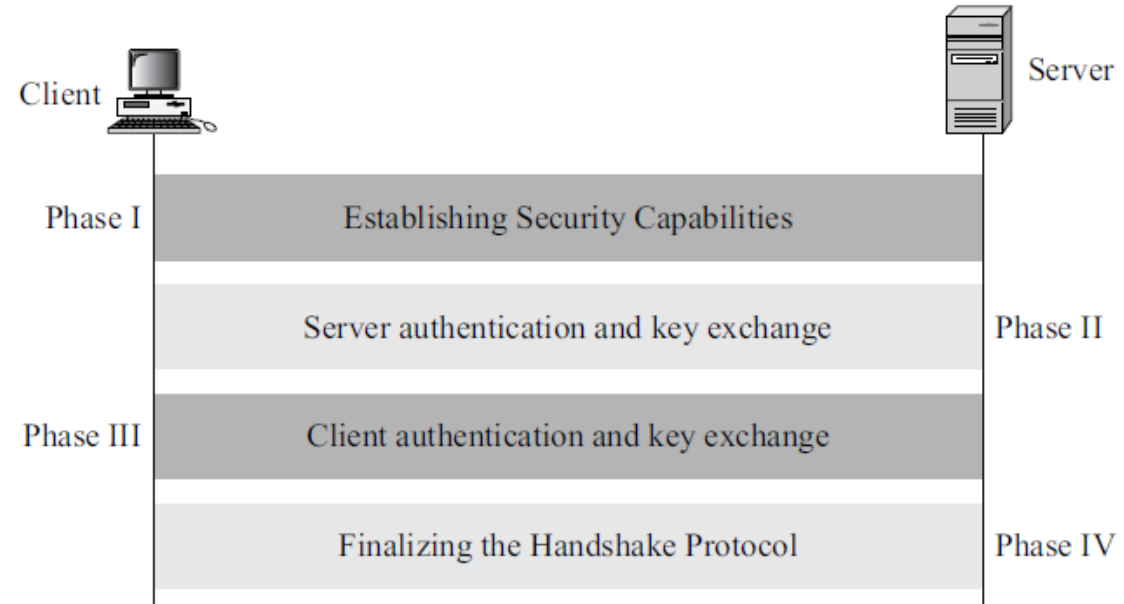


- SSL defines four protocols in two layers.
- The **Record Protocol** is the carrier. It carries messages from three other protocols as well as the data coming from the application layer. Messages from the Record Protocol are payloads to the transport layer, normally TCP.
- The **Handshake Protocol** provides security parameters for the Record Protocol. It establishes a cipher set and provides keys and security parameters. It also authenticates the server to the client and the client to the server if needed.
- The **ChangeCipherSpec** Protocol is used for signaling the readiness of cryptographic secrets.
- The **Alert Protocol** is used to report abnormal conditions.

Handshake protocol

- The **Handshake Protocol** uses messages to negotiate the cipher suite, to authenticate the server to the client and the client to the server if needed, and to exchange information for building the cryptographic secrets. The handshaking is done in four phases.

1. Establishing security capability
2. Server key exchange and authentication
3. Client key exchange and authentication
4. Finalizing and finishing



Handshake protocol

Phase I: Establishing Security Capability

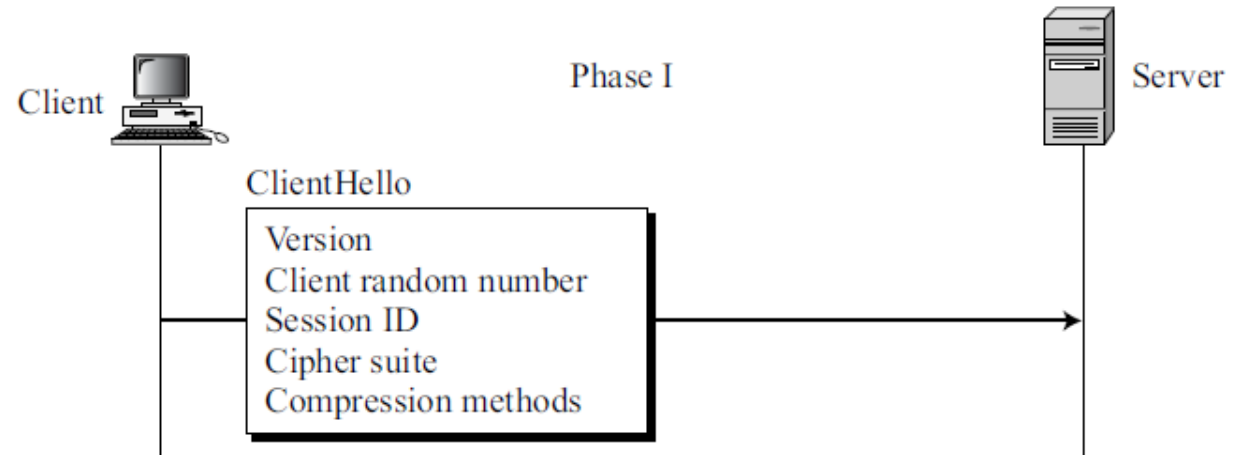
- In Phase I, the client and the server announce their security capabilities and choose those that are convenient for both.
- In this phase, a session ID is established, and the cipher suite is chosen.
- The parties agree upon a particular compression method.
- Finally, two random numbers are selected, one by the client and one by the server, to be used for creating a master secret.
- Two messages are exchanged in this phase: **ClientHello** and **ServerHello** messages.

Handshake protocol

Phase I: Establishing Security Capability

ClientHello message

- The client sends the **ClientHello** message. It contains the following:
 - The highest SSL version number the client can support.
 - A 32-byte random number (from the client) that will be used for master secret generation.
 - A session ID that defines the session.
 - A cipher suite that defines the list of algorithms that the client can support.
 - A list of compression methods that the client can support



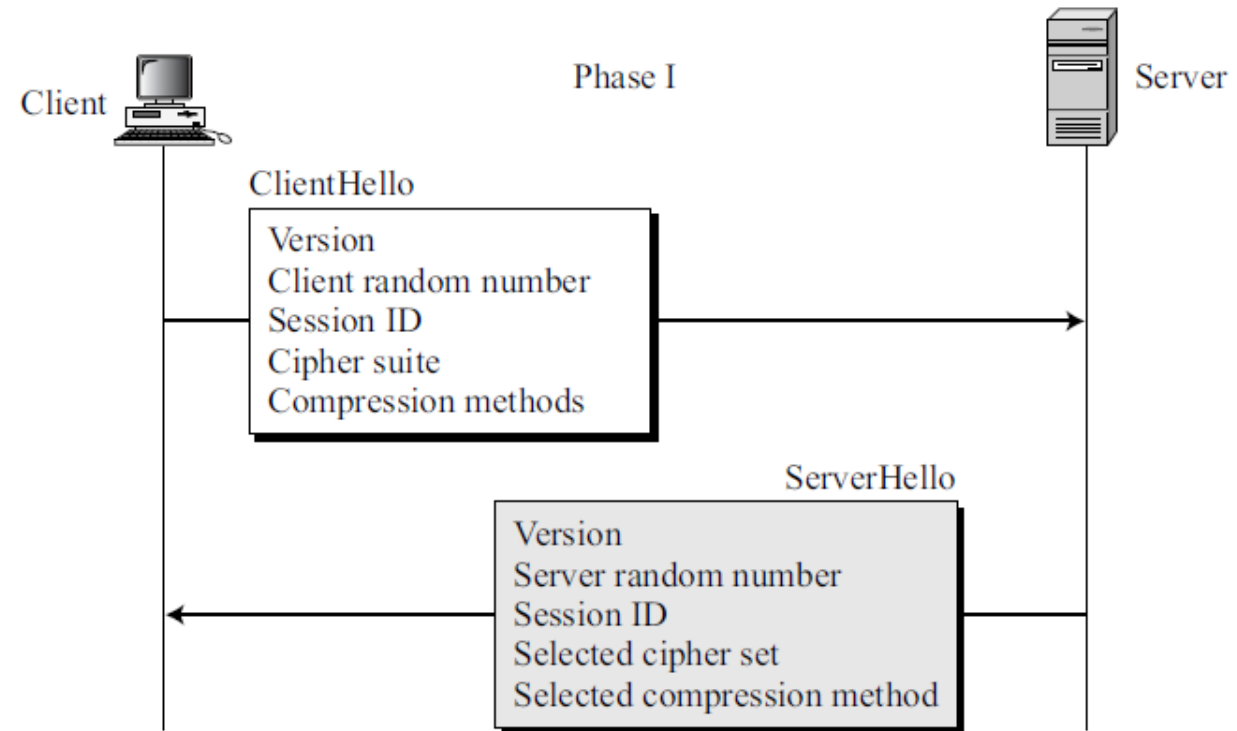
ClientHello message

Handshake protocol

Phase I: Establishing Security Capability

ServerHello message

- The server responds to the client with a **ServerHello** message. It contains the following:
 - The highest SSL version number the client can support.
 - A 32-byte random number (from the server) that will be used for master secret generation.
 - A session ID that defines the session.
 - The selected cipher set from the client list.
 - The selected compression method from the client list.



ServerHello message

Handshake protocol

At the culmination of Phase I,

Phase I: Establishing Security Capability

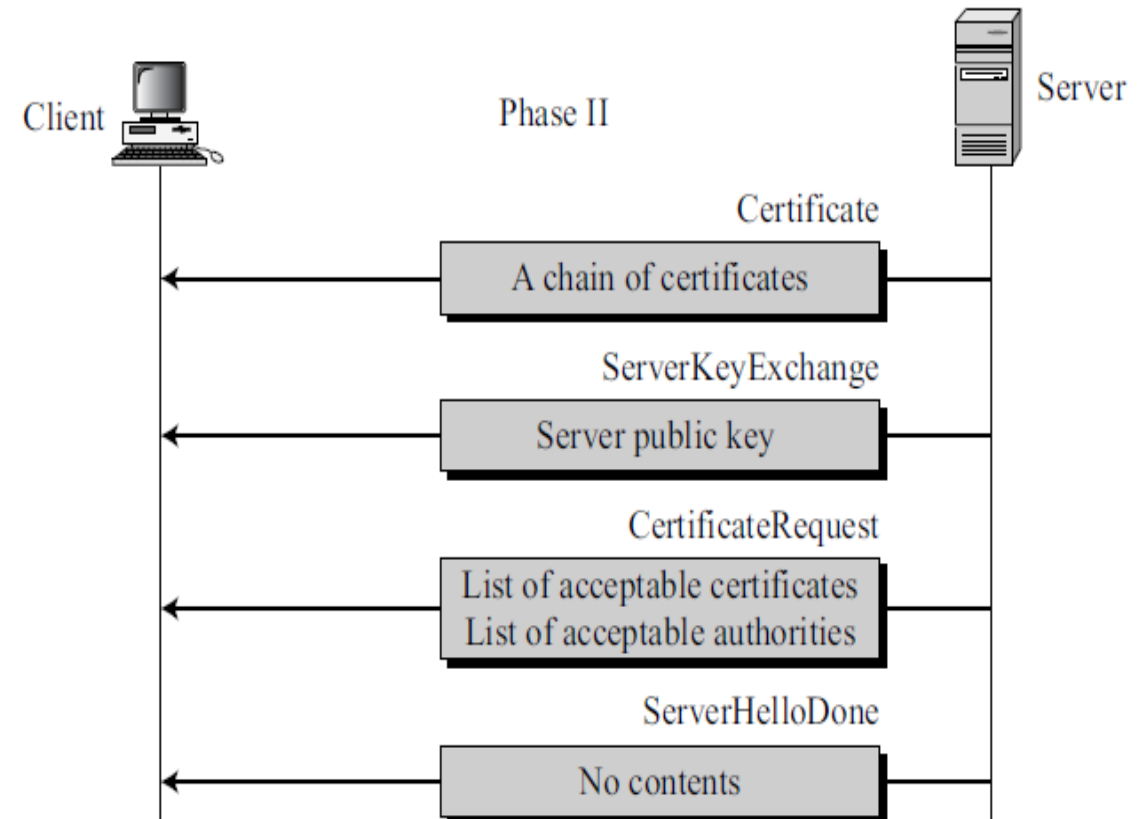
- After Phase I, the client and server know the following:
 - The version of SSL
 - The algorithms for key exchange, message authentication, and encryption
 - The compression method
 - The two random numbers for key generation

Handshake protocol

Phase II: Server Key Exchange and Authentication

- In phase II, the server authenticates itself if needed. The sender may send its certificate, its public key, and may also request certificates from the client. At the end, the server announces that the serverHello process is done.

- **Certificate** If it is required, the server sends a **Certificate** message to authenticate itself. The message includes a list of certificates of type X.509. The certificate is not needed if the key-exchange algorithm is anonymous Diffie-Hellman.
- **ServerKeyExchange** After the Certificate message, the server sends a **ServerKeyExchange** message that includes its contribution to the pre-master secret. This message is not required if the key-exchange method is RSA or fixed Diffie-Hellman.
- **CertificateRequest** The server may require the client to authenticate itself. In this case, the server sends a **CertificateRequest** message in Phase II that asks for certification in Phase III from the client. The server cannot request a certificate from the client if it is using anonymous Diffie-Hellman.
- **ServerHelloDone** The last message in Phase II is the ServerHelloDone message, which is a signal to the client that Phase II is over and that the client needs to start Phase III.



Handshake protocol

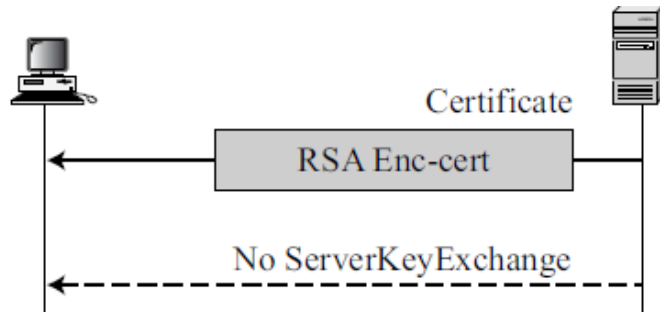
Phase II: Server Key Exchange and Authentication

At the culmination of Phase II,

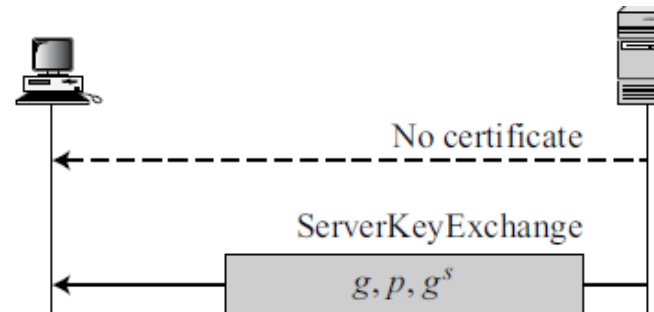
- After Phase II,
 - The server is authenticated to the client
 - The client knows the public key of the server if required

The first two messages in this phase are based on the key-exchange method

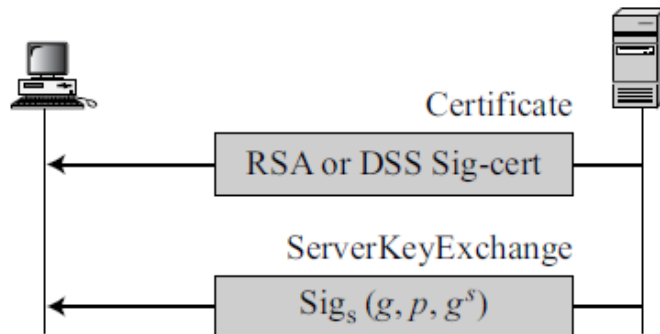
Four cases in Phase II



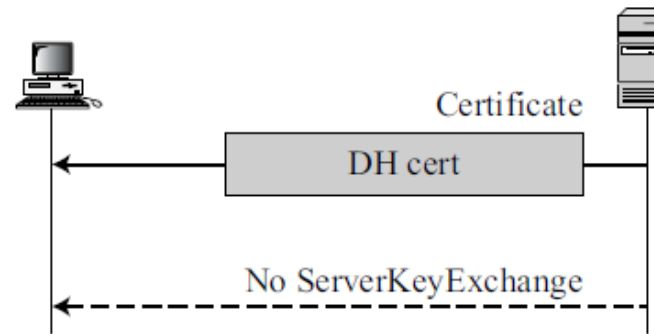
a. RSA



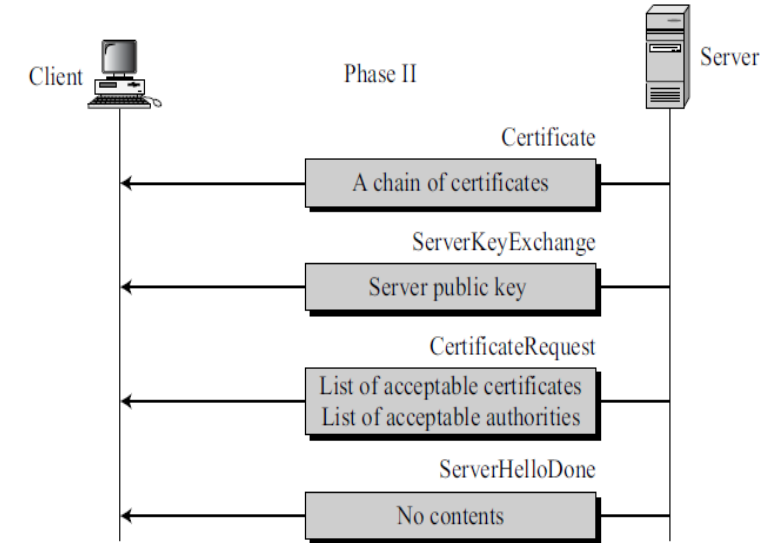
b. Anonymous DH



c. Ephemeral DH



d. Fixed DH

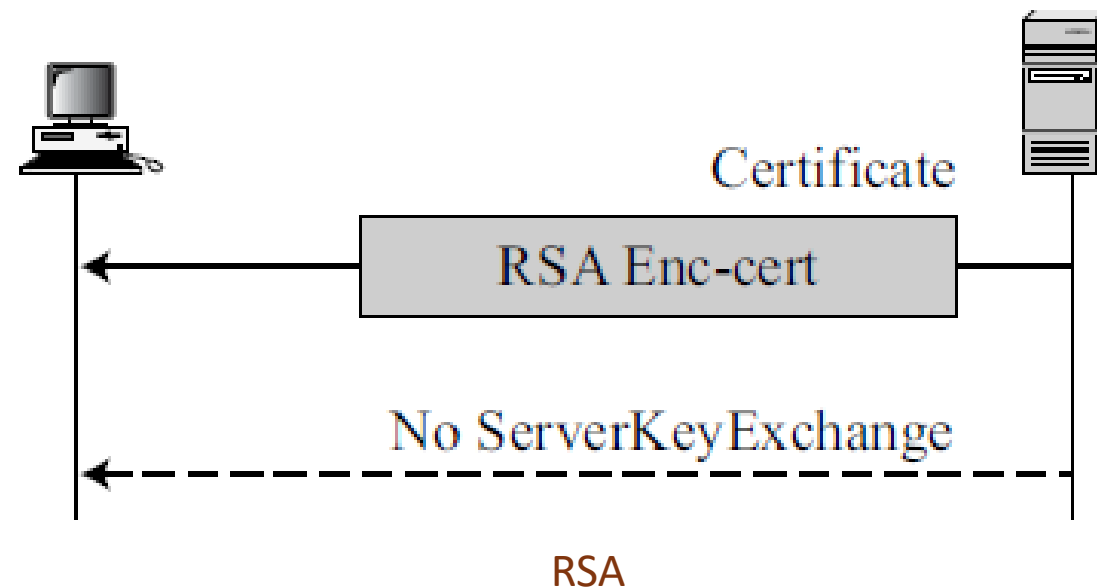


Phase II of handshake protocol

Handshake protocol

Phase II: Server Key Exchange and Authentication

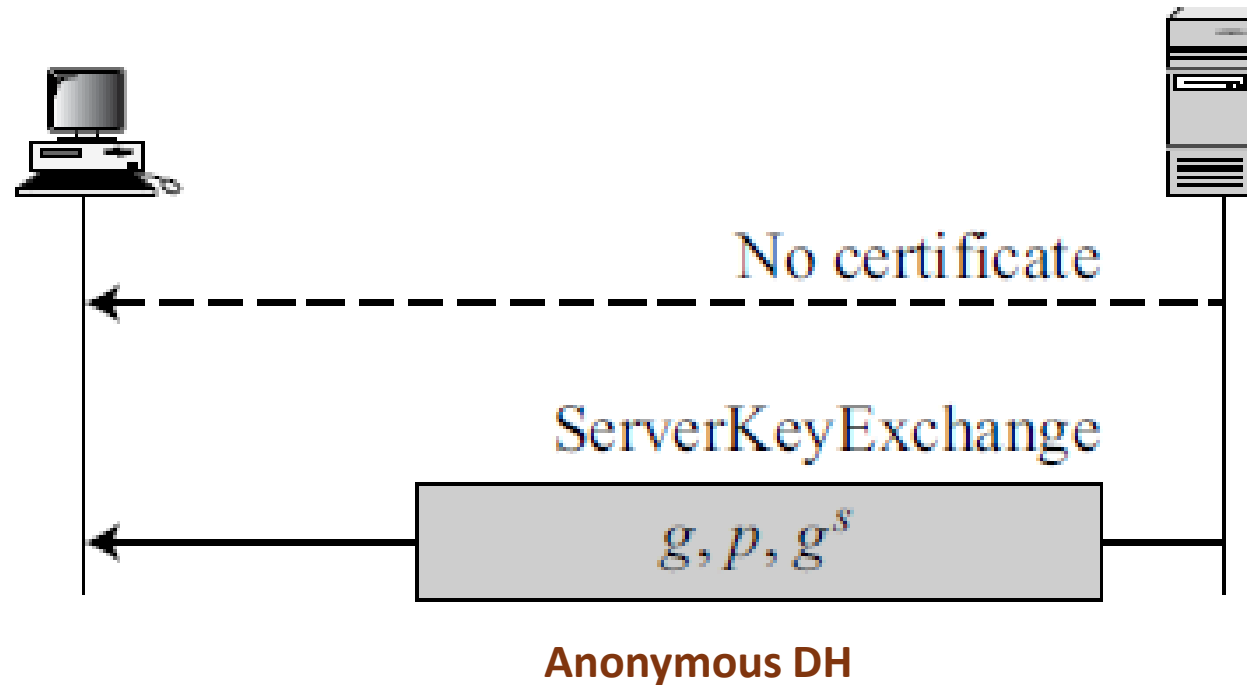
- **RSA:** In this method, the server sends its RSA encryption/decryption public-key certificate in the first message. The second message, however, is empty because the pre-master secret is generated and sent by the client in the next phase.
- The public-key certificate authenticates the server to the client. When the server receives the pre-master secret, it decrypts it with its private key. The possession of the private key by the server is proof that the server is the entity that it claims to be in the public-key certificate sent in the first message.



Handshake protocol

Phase II: Server Key Exchange and Authentication

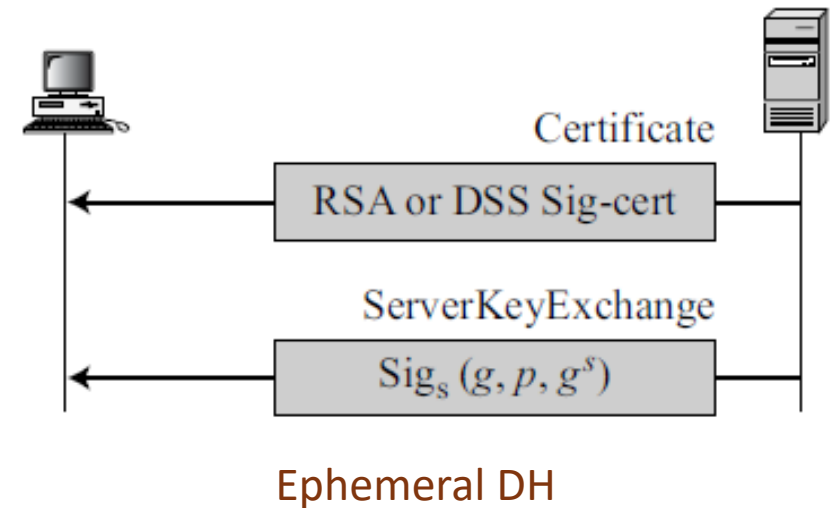
- **Anonymous DH:** In this method, there is no Certificate message. An anonymous entity does not have a certificate. In the ServerKeyExchange message, the server sends the Diffie-Hellman parameters and its half-key. Here, the server is not authenticated in this method.



Handshake protocol

Phase II: Server Key Exchange and Authentication

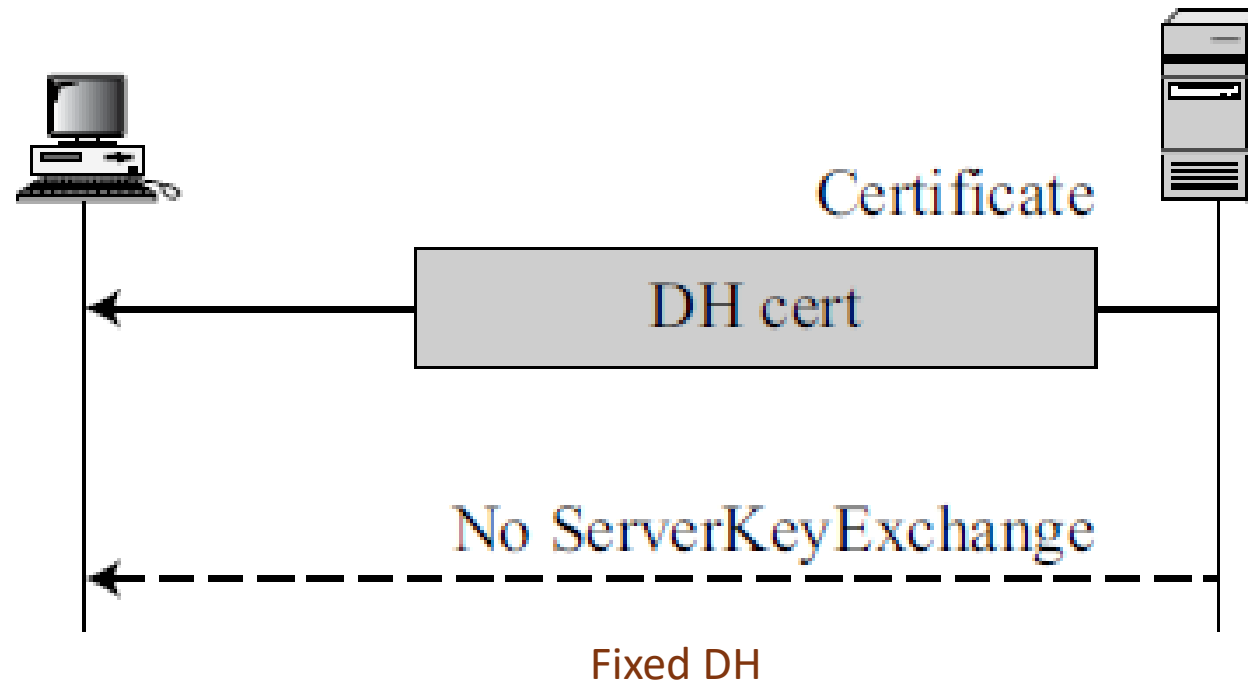
- **Ephemeral DH:** In this method, the server sends either an RSA or a DSS digital signature certificate.
- The private key associated with the certificate allows the server to sign a message; the public key allows the recipient to verify the signature.
- In the second message, the server sends the Diffie-Hellman parameters and the half-key signed by its private key. Other text is also sent. The server is authenticated to the client in this method, not because it sends the certificate, but because it signs the parameters and keys with its private key.
- The possession of the private key is proof that the server is the entity that it claims to be in the certificate.
- If an impostor copies and sends the certificate to the client, pretending that it is the server claimed in the certificate, it cannot sign the second message because it does not have the private key.



Handshake protocol

Phase II: Server Key Exchange and Authentication

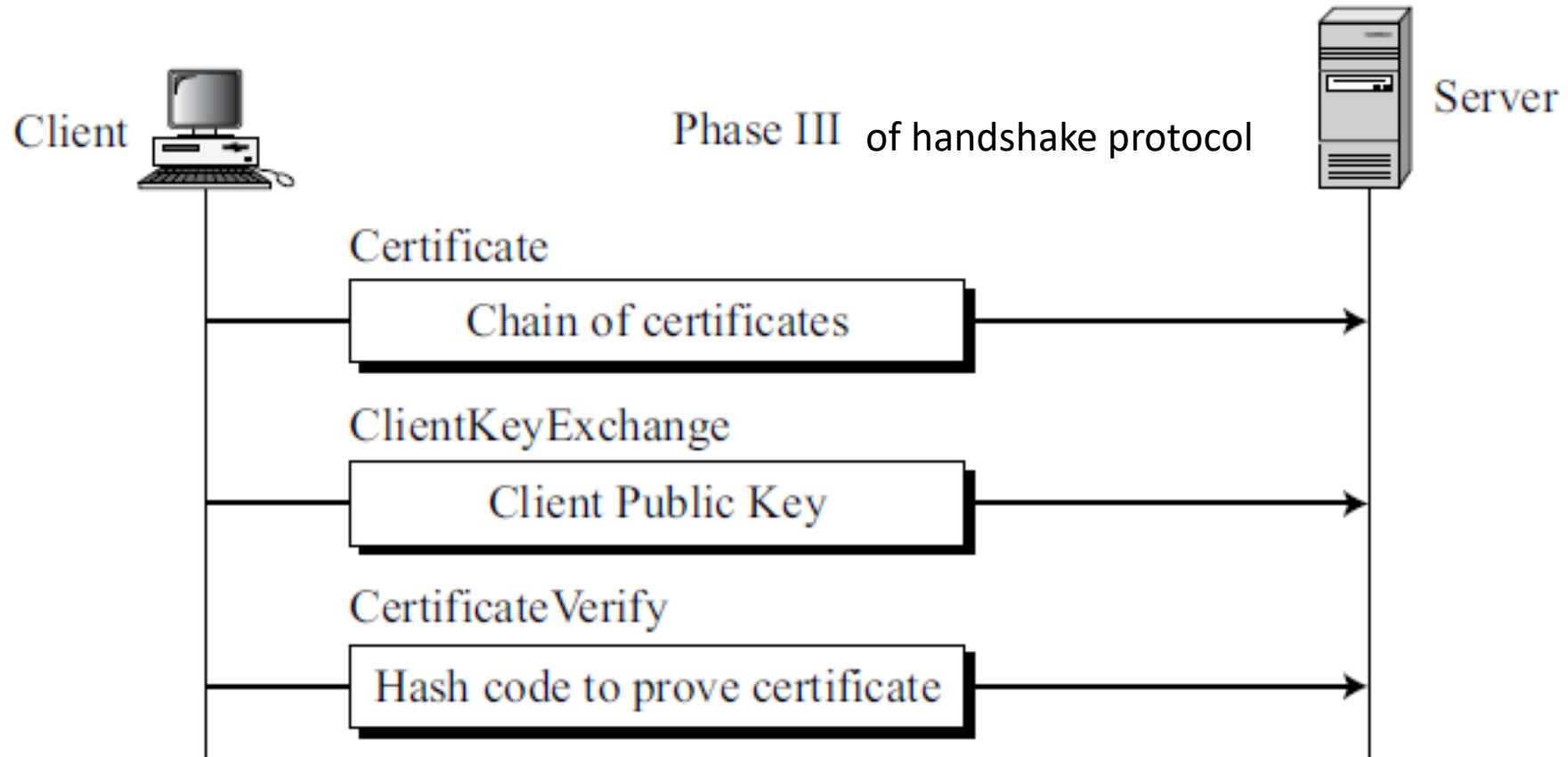
- **Fixed DH:** In this method, the server sends an RSA or DSS digital signature certificate that includes its registered DH half-key. The second message is empty.
- The certificate is signed by the CA's private key and can be verified by the client using the CA's public key. In other words, the CA is authenticated to the client and the CA claims that the half-key belongs to the server.



Handshake protocol

Phase III: Client key exchange and authentication

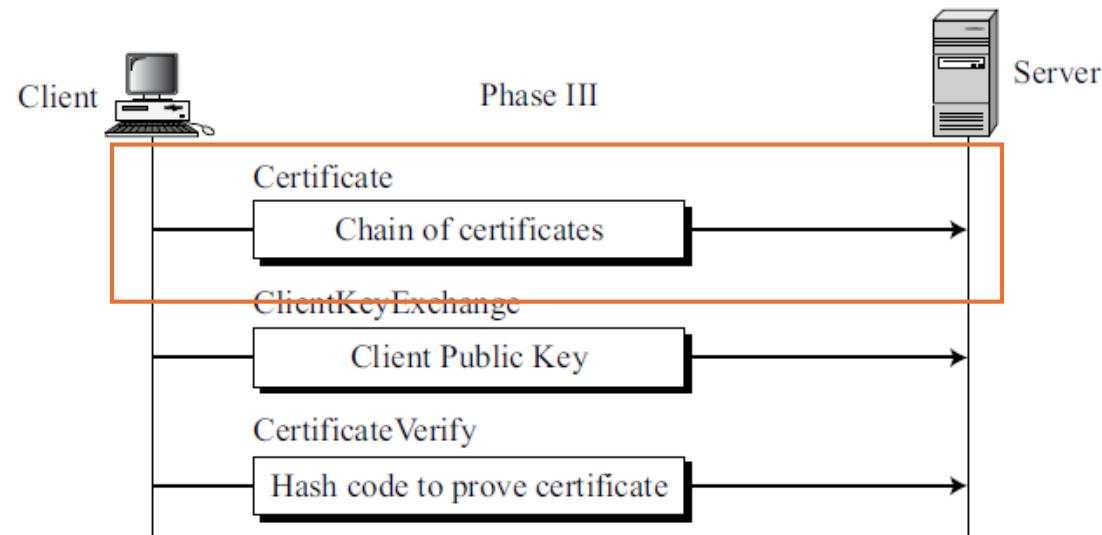
- Phase III is designed to authenticate the client. Up to three messages can be sent from the client to the server as shown in the figure.



Handshake protocol

Phase III: Client key exchange and authentication

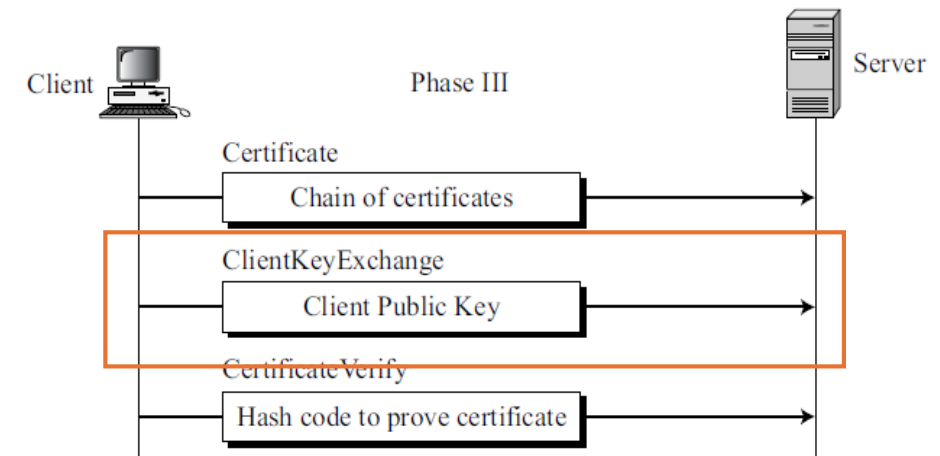
- **Certificate** To certify itself to the server, the client sends a Certificate message. Note that the format is the same as the Certificate message sent by the server in Phase II, but the contents are different. It includes the chain of certificates that certify the client. This message is sent only if the server has requested a certificate in Phase II. If there is a request and the client has no certificate to send, it sends an Alert message (part of the Alert Protocol to be discussed later) with a warning that there is no certificate. The server may continue with the session or may decide to abort.



Handshake protocol

Phase III: Client key exchange and authentication

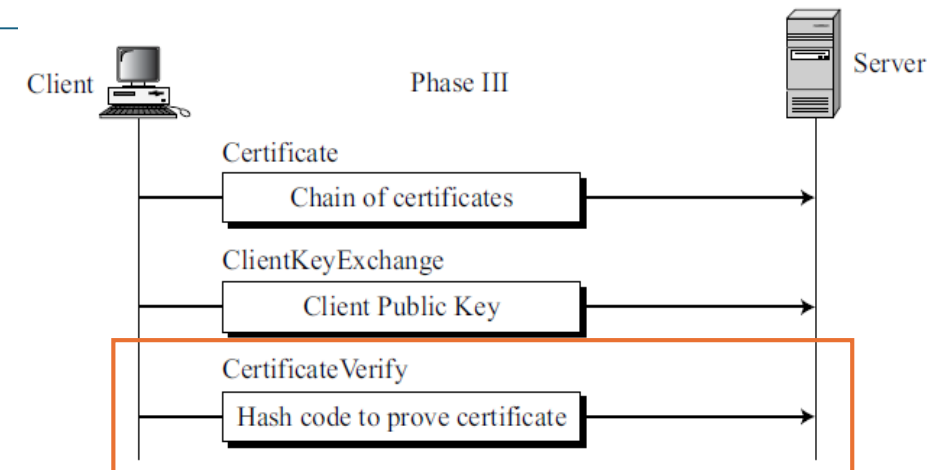
- **ClientKeyExchange** After sending the Certificate message, the client sends a ClientKeyExchange message, which includes its contribution to the pre-master secret. The contents of this message are based on the key-exchange algorithm used. If the method is RSA, the client creates the entire pre-master secret and encrypts it with the RSA public key of the server. If the method is anonymous or ephemeral Diffie-Hellman, the client sends its Diffie-Hellman half-key. If the method is Fortezza, the client sends the Fortezza parameters. The contents of this message are empty if the method is fixed Diffie-Hellman.



Handshake protocol

Phase III: Client key exchange and authentication

- **CertificateVerify** If the client has sent a certificate declaring that it owns the public key in the certificate, it needs to prove that it knows the corresponding private key. This is needed to thwart an impostor who sends the certificate and claims that it comes from the client. The proof of private-key possession is done by creating a message and signing it with the private key. The server can verify the message with the public key already sent to ensure that the certificate actually belongs to the client. this is possible if the certificate has a signing capability; a pair of keys, public and private, is involved. The certificate for fixed Diffie-Hellman cannot be verified this way.



Handshake protocol

Phase III: Client key exchange and authentication

At the culmination of Phase III,

- After Phase III,
 - The client is authenticated to server.
 - Both the client and the server know the pre-master secret.

The three messages in this phase are based on the key-exchange method.

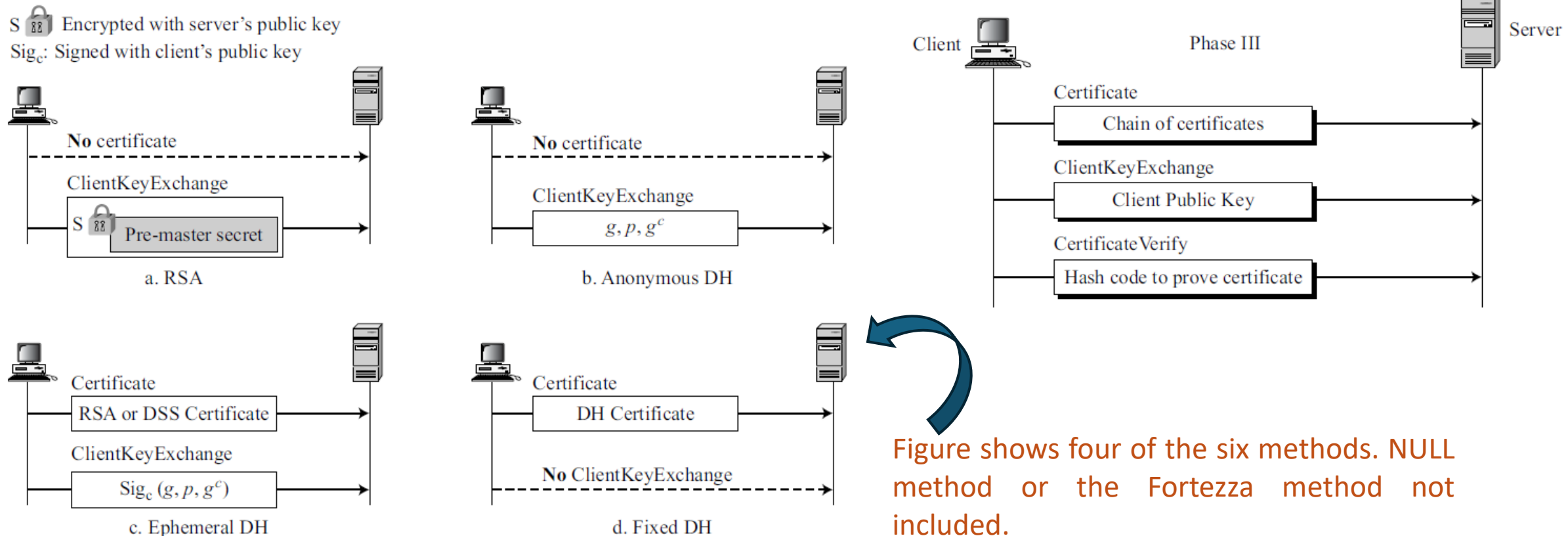
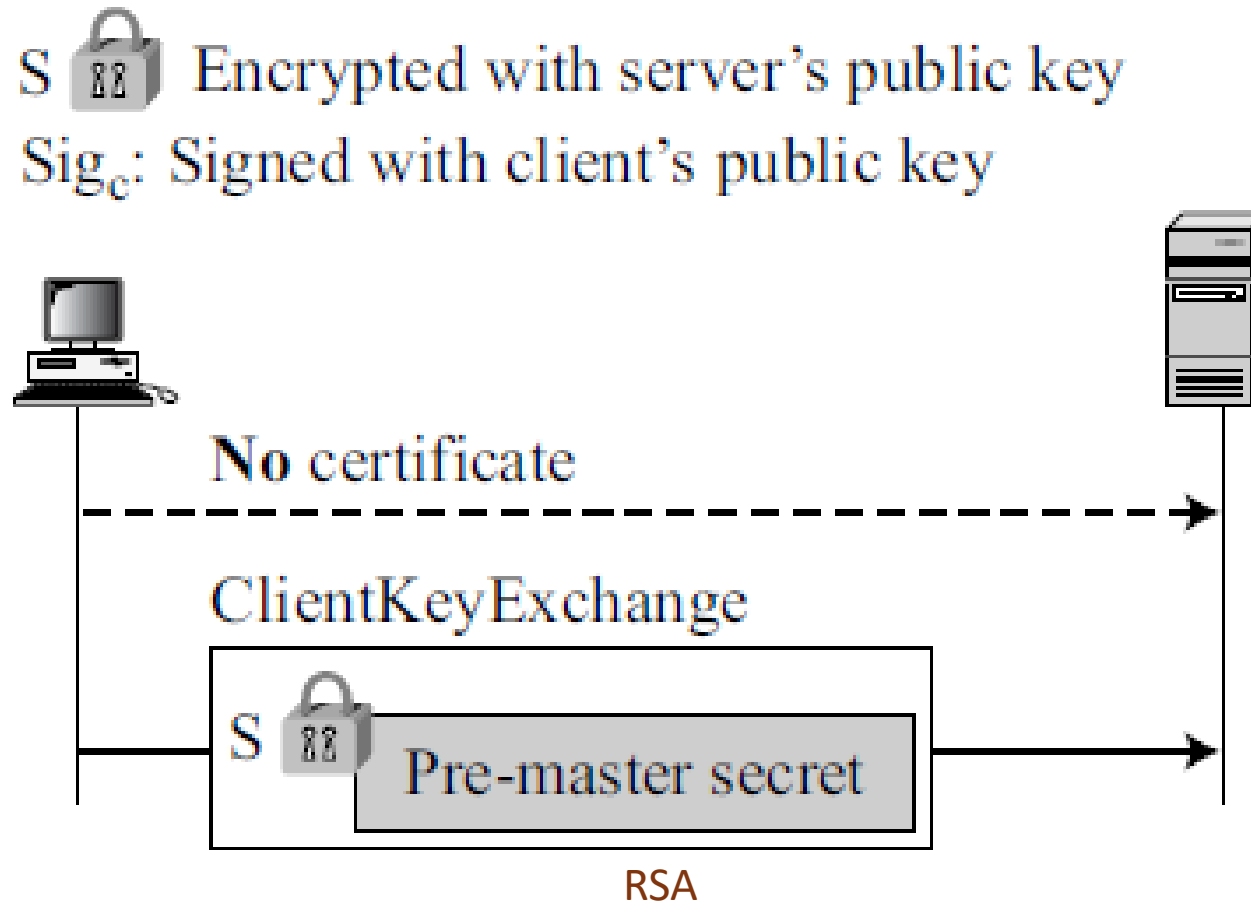


Figure shows four of the six methods. NULL method or the Fortezza method not included.

Handshake protocol

Phase III: Client key exchange and authentication

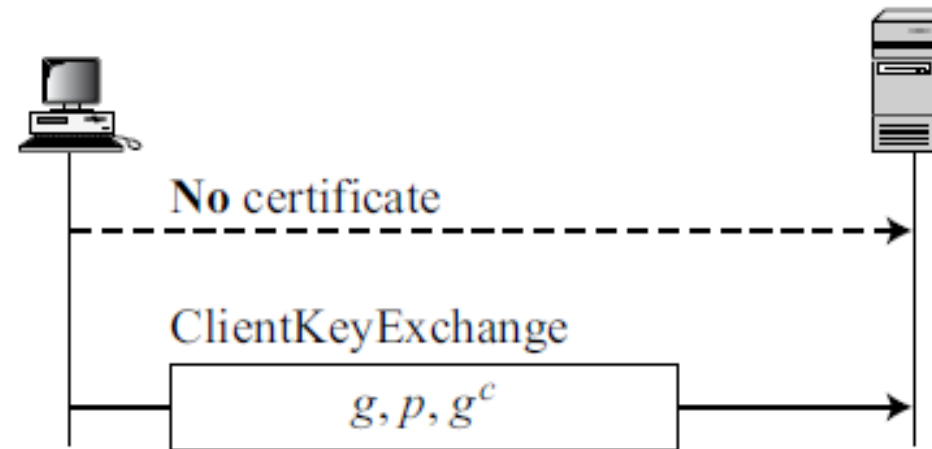
- **RSA:** In this case, there is no Certificate message unless the server has explicitly requested one in Phase II. The ClientKeyExchange method includes the pre-master key encrypted with the RSA public key received in Phase II.



Handshake protocol

Phase III: Client key exchange and authentication

- **Anonymous DH:** In this method, there is no Certificate message. The server does not have the right to ask for the certificate (in Phase II) because both the client and the server are anonymous. In the ClientKeyExchange message, the server sends the Diffie-Hellman parameters and its half-key. Note that the client is not authenticated to the server in this method.

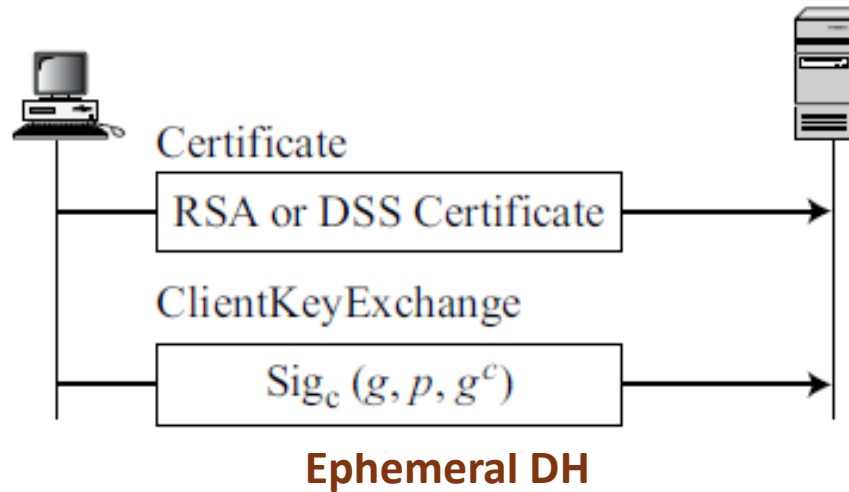


Anonymous DH

Handshake protocol

Phase III: Client key exchange and authentication

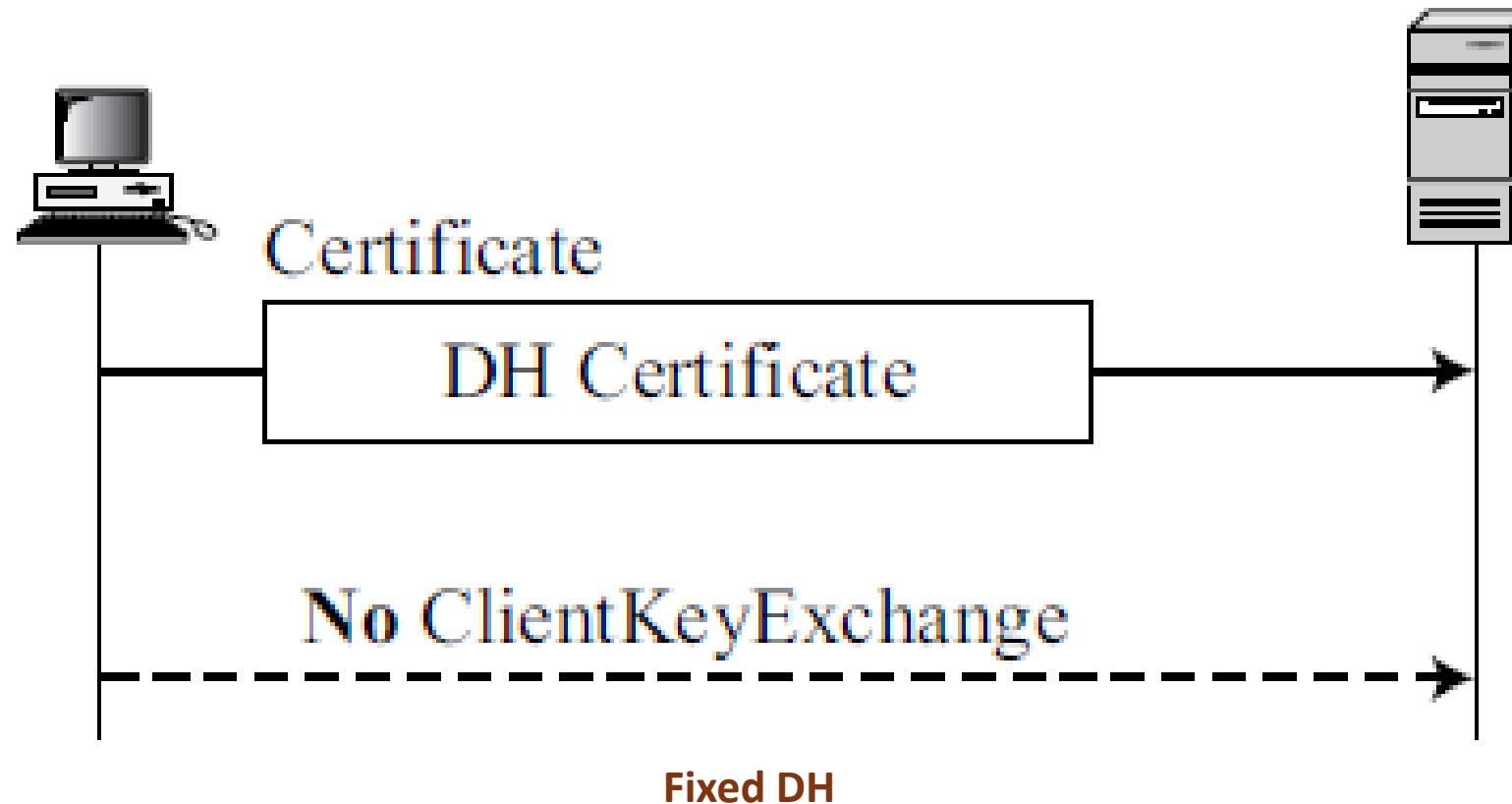
- **Ephemeral DH:** In this method, the client usually has a certificate. The server needs to send its RSA or DSS certificate (based on the agreed-upon cipher set). In the ClientKeyExchange message, the client signs the DH parameters and its halfkey and sends them. The client is authenticated to the server by signing the second message. If the client does not have the certificate, and the server asks for it, the client sends an Alert message to warn the client. If this is acceptable to the server, the client sends the DH parameters and key in plaintext. Of course, the client is not authenticated to the server in this situation.



Handshake protocol

Phase III: Client key exchange and authentication

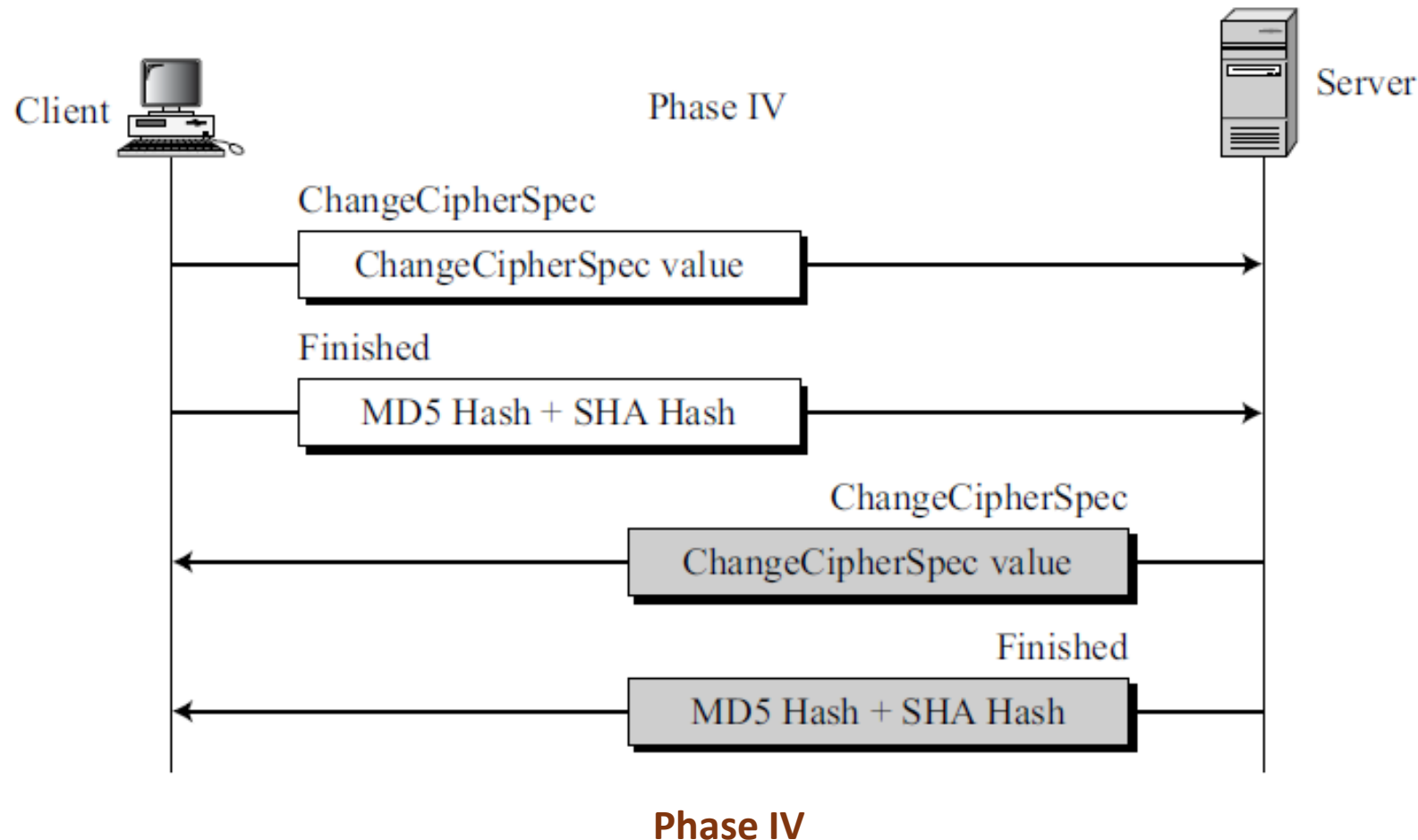
- **Fixed DH:** In this method, the client usually sends a DH certificate in the first message. Note that the second message is empty in this method. The client is authenticated to the server by sending the DH certificate.



Handshake protocol

Phase IV: Finalizing and finishing

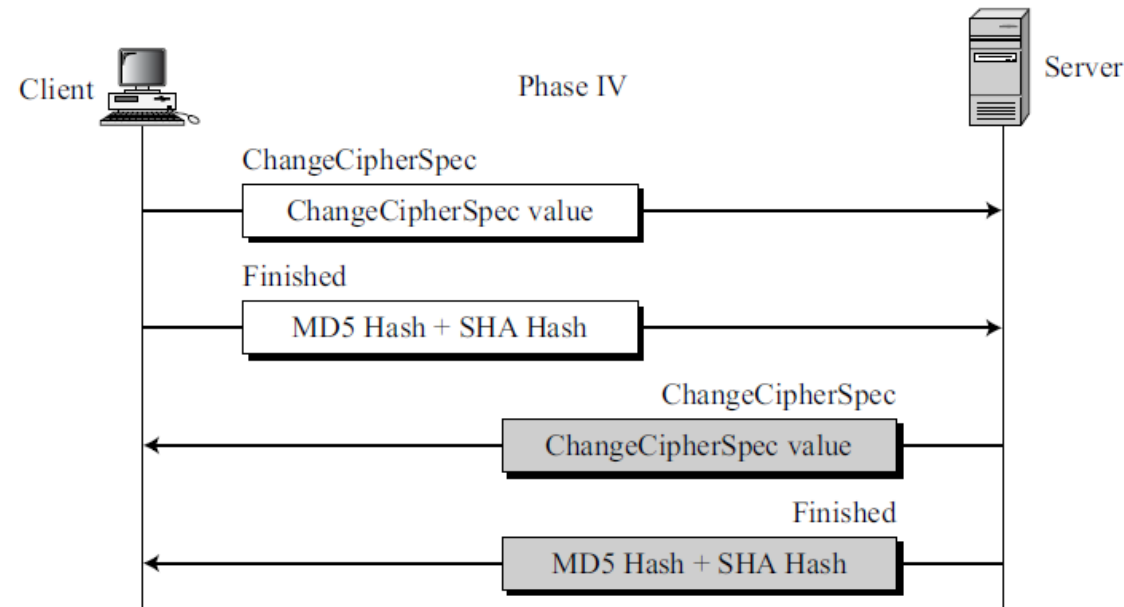
- In Phase IV, the client and server send messages to change cipher specification and to finish the handshaking protocol. Four messages are exchanged in this phase.



Handshake protocol

Phase IV: Finalizing and finishing

- The client sends a **ChangeCipherSpec** message to show that it has moved all of the cipher suite set and the parameters from the pending state to the active state.
- The next message is also sent by the client. It is a **Finished** message that announces the end of the handshaking protocol by the client.
- The server sends a **ChangeCipherSpec** message to show that it has also moved all of the cipher suite set and parameters from the pending state to the active state.
- Finally, the server sends a **Finished** message to show that handshaking is totally completed.



ChangeCipherSpec protocol

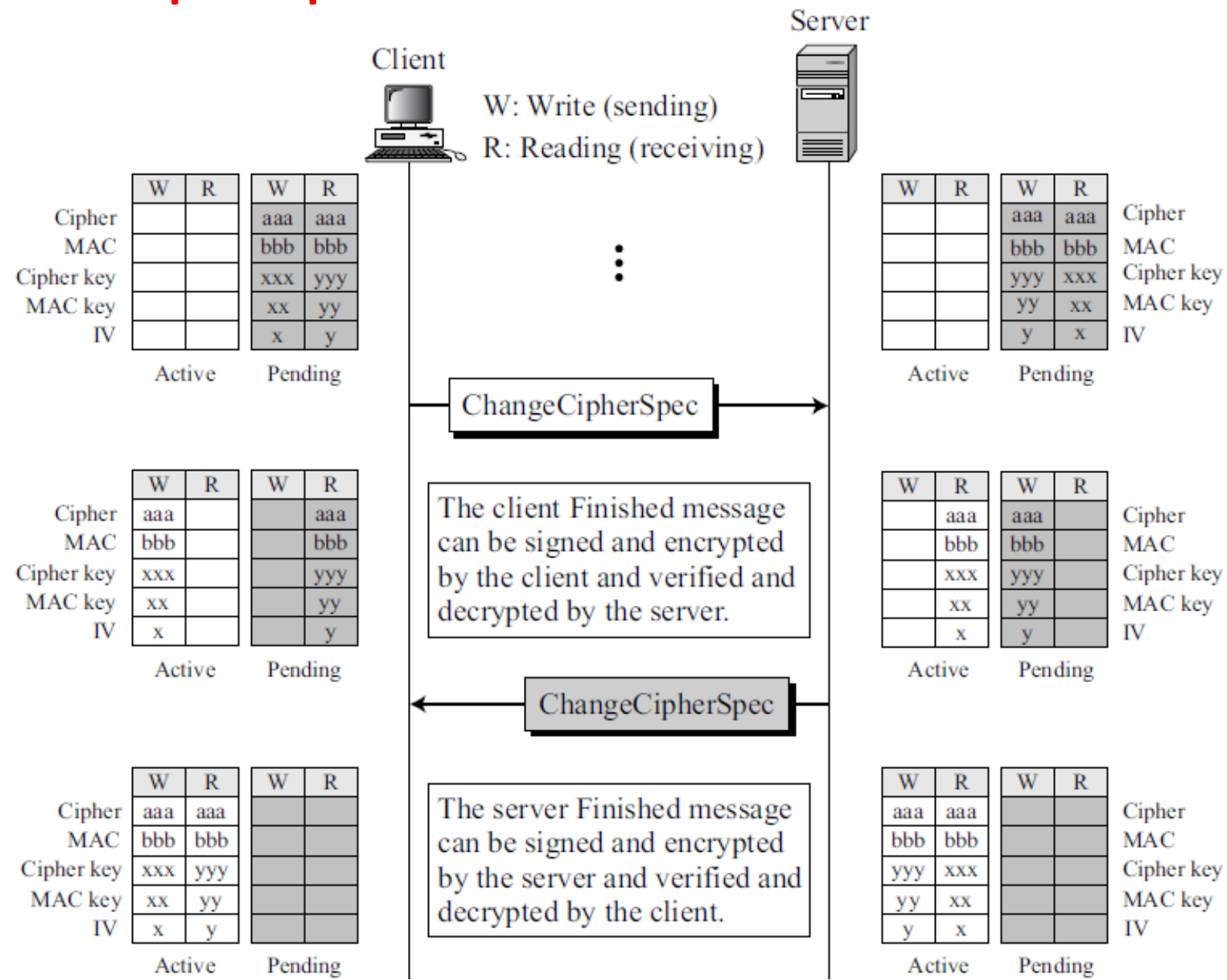
- The negotiation of the cipher suite and the generation of cryptographic secrets are formed gradually during the Handshake Protocol.
- **The question now is:** When can the two parties use these parameter secrets.
- SSL mandates that the parties cannot use these parameters or secrets until they have sent or received a special message, the **ChangeCipherSpec** message, which is exchanged during the Handshake protocol and defined in the **ChangeCipherSpec** Protocol.
- The reason is that the issue is not just sending or receiving a message. The sender and the receiver need two states, not one. One state, the pending state, keeps track of the parameters and secrets.
- The other state, the active state, holds parameters and secrets used by the Record Protocol to sign/verify or encrypt/decrypt messages. In addition, each state holds two sets of values: read (inbound) and write (outbound). The **ChangeCipherSpec** Protocol defines the process of moving values between the pending and active states.

ChangeCipherSpec protocol

- The following figure shows a hypothetical situation, with hypothetical values, that explains the concept. Only a few parameters are shown. Before the exchange of any **ChangeCipherSpec** messages, only the pending columns have values.
- First the client sends a **ChangeCipherSpec** message. After the client sends this message, it moves the write (outbound) parameters from pending to active. The client can now use these parameters to sign or encrypt outbound messages. After the receiver receives this message, it moves the read (inbound) parameters from the pending to the active state. Now the server can verify and decrypt messages. This means that the Finished message sent by the client can be signed and encrypted by the client and verified and decrypted by the server.
- The server sends the **ChangeCipherSpec** message after receiving the Finish message from the client. After sending this message it moves the write (outbound) parameters from pending to active. The server can now use these parameters to sign or encrypt outbound messages. After the client receives this message, it moves the read (inbound) parameters from the pending to the active state. Now the client can verify and decrypt messages. after the exchanged Finished messages, both parties can communicate in both directions using the read/write active parameters.

ChangeCipherSpec protocol

Movement of parameters from pending state to active state



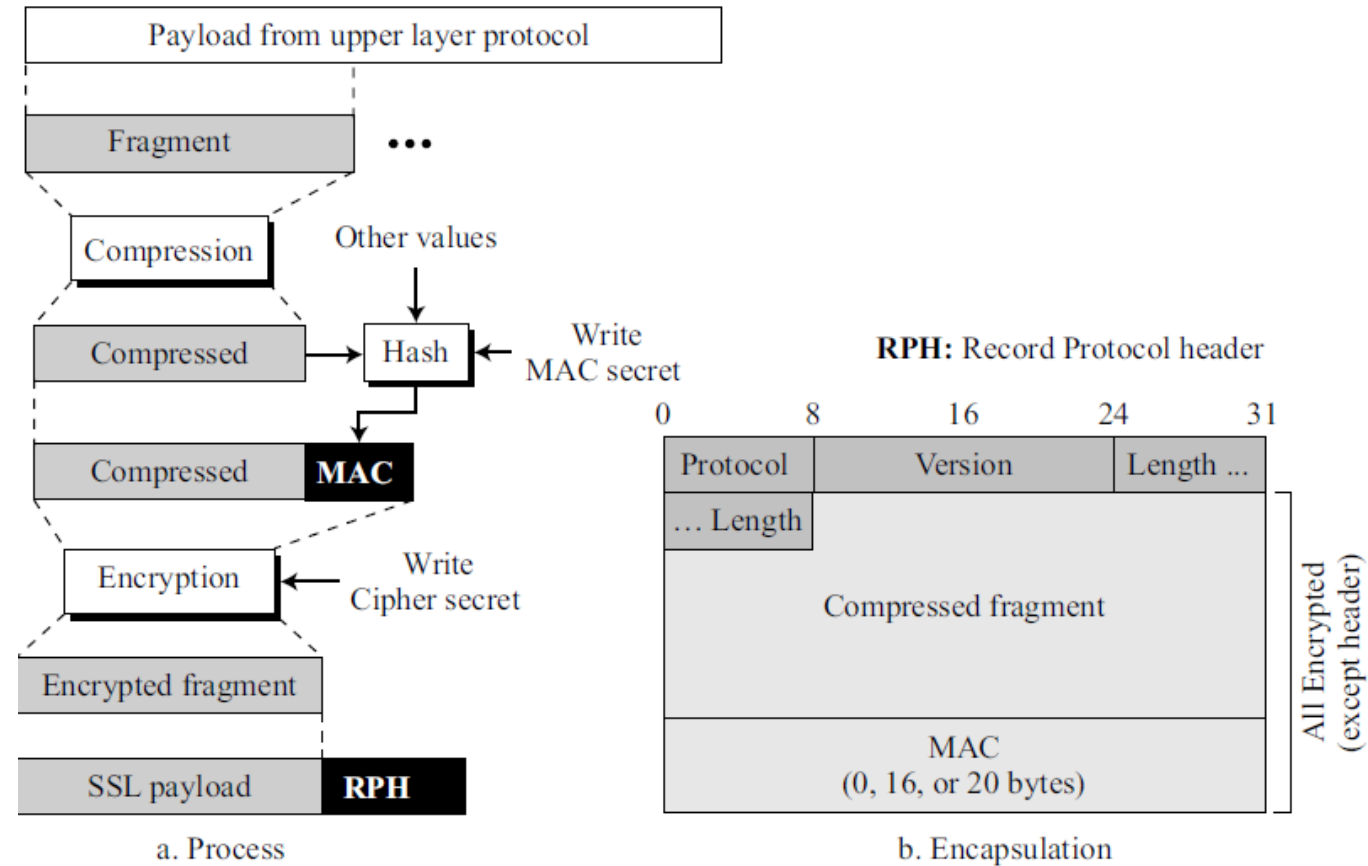
Alert protocol

- SSL uses the Alert Protocol for reporting errors and abnormal conditions.
- It has only one message type, the Alert message, that describes the problem and its level (warning or fatal). Table shows the types of Alert messages defined for SSL.

<i>Value</i>	<i>Description</i>	<i>Meaning</i>
0	<i>CloseNotify</i>	Sender will not send any more messages.
10	<i>UnexpectedMessage</i>	An inappropriate message received.
20	<i>BadRecordMAC</i>	An incorrect MAC received.
30	<i>DecompressionFailure</i>	Unable to decompress appropriately.
40	<i>HandshakeFailure</i>	Sender unable to finalize the handshake.
41	<i>NoCertificate</i>	Client has no certificate to send.
42	<i>BadCertificate</i>	Received certificate corrupted.
43	<i>UnsupportedCertificate</i>	Type of received certificate is not supported.
44	<i>CertificateRevoked</i>	Signer has revoked the certificate.
45	<i>CertificateExpired</i>	Certificate expired.
46	<i>CertificateUnknown</i>	Certificate unknown.
47	<i>IllegalParameter</i>	An out-of-range or inconsistent field.

Record protocol

- The **Record Protocol** carries messages from the upper layer (Handshake Protocol, ChangeCipherSpec Protocol, Alert Protocol, or application layer).
- The message is fragmented and optionally compressed; a MAC is added to the compressed message using the negotiated hash algorithm.
- The compressed fragment and the MAC are encrypted using the negotiated encryption algorithm. Finally, the SSL header is added to the encrypted message. The figure shows this process at the sender. The process at the receiver is reversed. Note that this process can only be done when the cryptographic parameters are in the active state.
- Messages sent before the movement from pending to active are neither signed nor encrypted.



Processing done by the Record Protocol

Record protocol

- **Fragmentation/Combination**

- At the sender, a message from the application layer is fragmented into blocks of 2^{14} bytes, with the last block possibly less than this size. At the receiver, the fragments are combined together to make a replica of the original message.

- **Compression/decompression**

- At the sender, all application layer fragments are compressed by the compression method negotiated during the handshaking. The compression method needs to be lossless (the decompressed fragment must be an exact replica of the original fragment). The size of the fragment must not exceed 1024 bytes.
- Some compression methods work only on a predefined block size and if the size of the block is less than this, some padding is added. Therefore, the size of the compressed fragment may be greater than the size of the original fragment.
- At the receiver, the compressed fragment is decompressed to create a replica of the original. If the size of the decompressed fragment exceeds 2^{14} , a fatal decompression Alert message is issued. Note that compression/decompression is optional in SSL.

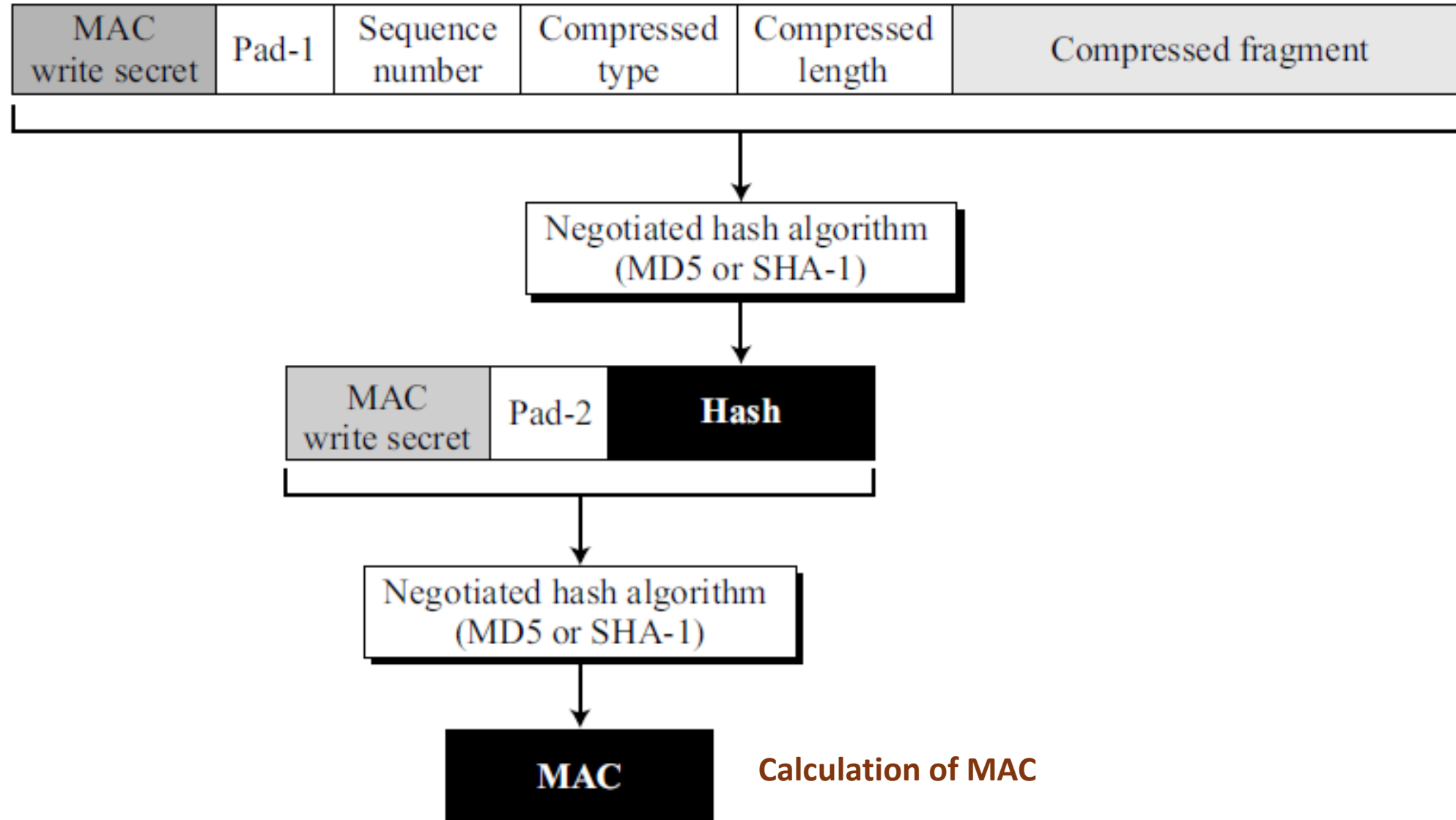
- **Signing/Verifying**

- At the sender, the authentication method defined during the handshake (NULL, MD5, or SHA-1) creates a signature (MAC) as shown in the figure. (next slide)

Record protocol

Pad-1: Byte 0x36 (00110110) repeated 48 times for MD5 and 40 times for SHA-1

Pad-2: Byte 0x5C (01011100) repeated 48 times for MD5 and 40 times for SHA-1



Record protocol

- The hash algorithm is applied twice. First, a hash is created from the concatenations of the following values.
- a. The MAC write secret (authentication key for the outbound message)
- b. Pad-1, which is the byte 0x36 repeated 48 times for MD5 and 40 times for SHA-1
- c. The sequence number for this message.
- d. The compressed type, which defines the upper-layer protocol that provided the compressed fragment.
- e. The compressed length, which is the length of the compressed fragment.
- f. The compressed fragment itself.

Second, the final hash (MAC) is created from the concatenation of the following values:

1. The MAC write secret.
2. Pad-2, which is the byte 0x5C repeated 48 times for MD5 and 40 times for SHA-1.
3. The hash created from the first step.

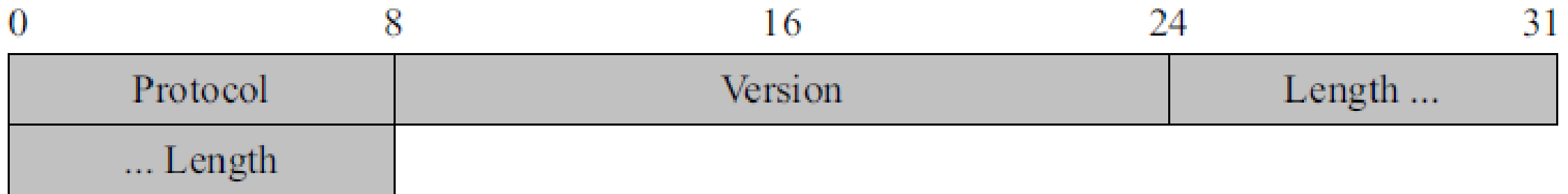
At the receiver, the verifying is done by calculating a new hash and comparing it to the received hash.

Encryption/Decryption: At the sender, the compressed fragment and the hash are encrypted using the cipher write secret. At the receiver, the received message is decrypted using the cipher read secret. For block encryption, padding is added to make the size of the encryptable message a multiple of the block size.

Framing/De-framing: After the encryption, the Record Protocol header is added at the sender. The header is removed at the receiver before decryption.

SSL message formats

- The messages from three protocols and data from the application layer are encapsulated in the Record Protocol messages. In other words, the Record Protocol message encapsulates messages from four different sources at the sender site. At the receiver site, the Record Protocol decapsulates the messages and delivers them to different destinations.
- The Record Protocol has a general header that is added to each message coming from the sources. Record Protocol general header.

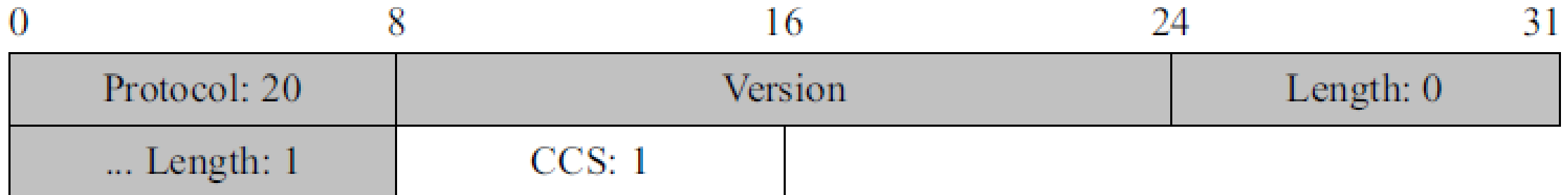


SSL message formats

- **Protocol** This 1-byte field defines the source or destination of the encapsulated message. It is used for multiplexing and demultiplexing. The values are 20 (**ChangeCipherSpec** Protocol), 21 (**Alert Protocol**), 22 (**Handshake Protocol**), and 23 (**data from the application layer**).
- **Version** This 2-byte field defines the version of the SSL; one byte is the major version, and the other is the minor. The current version of SSL is 3.0 (major 3 and minor 0).
- **Length** This 2-byte field defines the size of the message (without the header) in bytes.

ChangeCipherSpec Protocol

- The ChangeCipherSpec Protocol has one message, the **ChangeCipherSpec** message. The message is only one byte, encapsulated in the Record Protocol message with protocol value 20.
- The one-byte field in the message is called the CCS and its value is currently 1.



Alert protocol

- The Alert Protocol, as we discussed before, has one message that reports errors in the process. Figure shows the encapsulation of this single message in the Record Protocol with protocol value 21.
- The two fields of the Alert message are listed below.
- **Level.** This one-byte field defines the level of the error. Two levels have been defined so far: warning and fatal.
- **Description.** The one-byte description defines the type of error.

0	8	16	24	31
Protocol: 21	Version		Length: 0	
... Length: 2	Level	Description		

Handshake Protocol

- Several messages have been defined for the Handshake Protocol. All of these messages have the four-byte generic header.
- The figure shows the Record Protocol header and the generic header for the Handshake Protocol. Note that the value of the protocol field is 22.
- **Type:** This one-byte field defines the type of message. So far ten types have been defined as listed in Table.

0	8	16	24	31
Protocol: 22		Version		Length: ...
... Length:		Type:	Len ...	
... Len:		Generic header for Handshake Protocol		

Length (Len): This three-byte field defines the length of the message (excluding the length of the type and length field). The reader may wonder why we need two length fields, one in the general Record header and one in the generic header for the Handshake messages. The answer is that a Record message may carry two Handshake messages at the same time if there is no need for another message in between.

Type	Message
0	HelloRequest
1	ClientHello
2	ServerHello
11	Certificate
12	ServerKeyExchange
13	CertificateRequest
14	ServerHelloDone
15	CertificateVerify
16	ClientKeyExchange
20	Finished

Types of Handshake messages

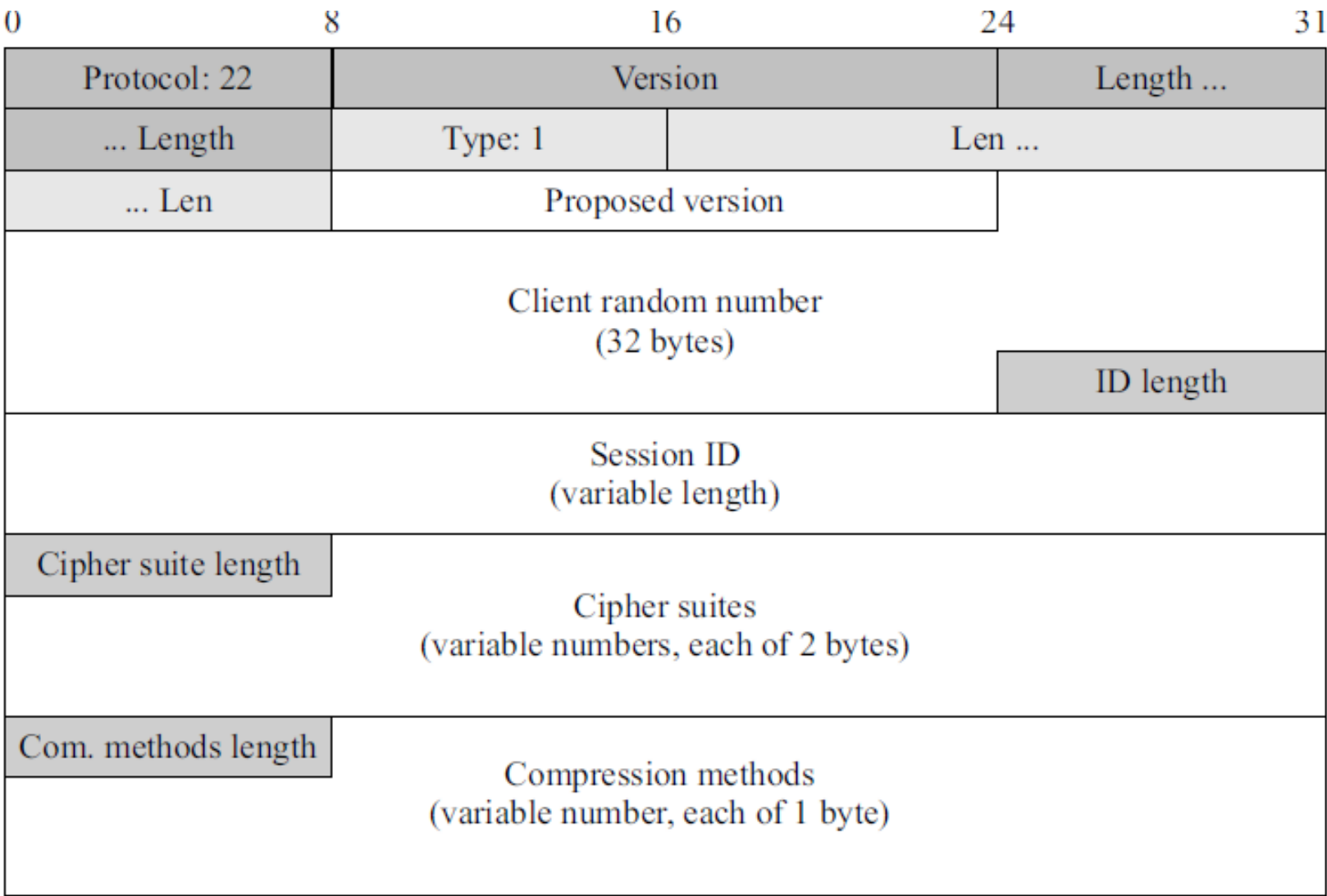
HelloRequest Message

- The HelloRequest message, which is rarely used, is a request from the server to the client to restart a session. This may be needed if the server feels that something is wrong with the session and a fresh session is needed. For example, if the session becomes so long that it threatens the security of the session, the server may send this message. The client then needs to send a ClientHello message and negotiate the security parameters. The below figure shows the format of this message. It is four bytes with a type value of 0. The message has no body, so the value of the length field is also 0.

0	8	16	24	31
Protocol: 22	Version			Length ...
... Length: 4	Type: 0	Len ...		
... Len: 0				

ClientHello Message

- The ClientHello message is the first message exchanged during handshaking. The figure shows the format of this message.



ClientHello Message cntd..

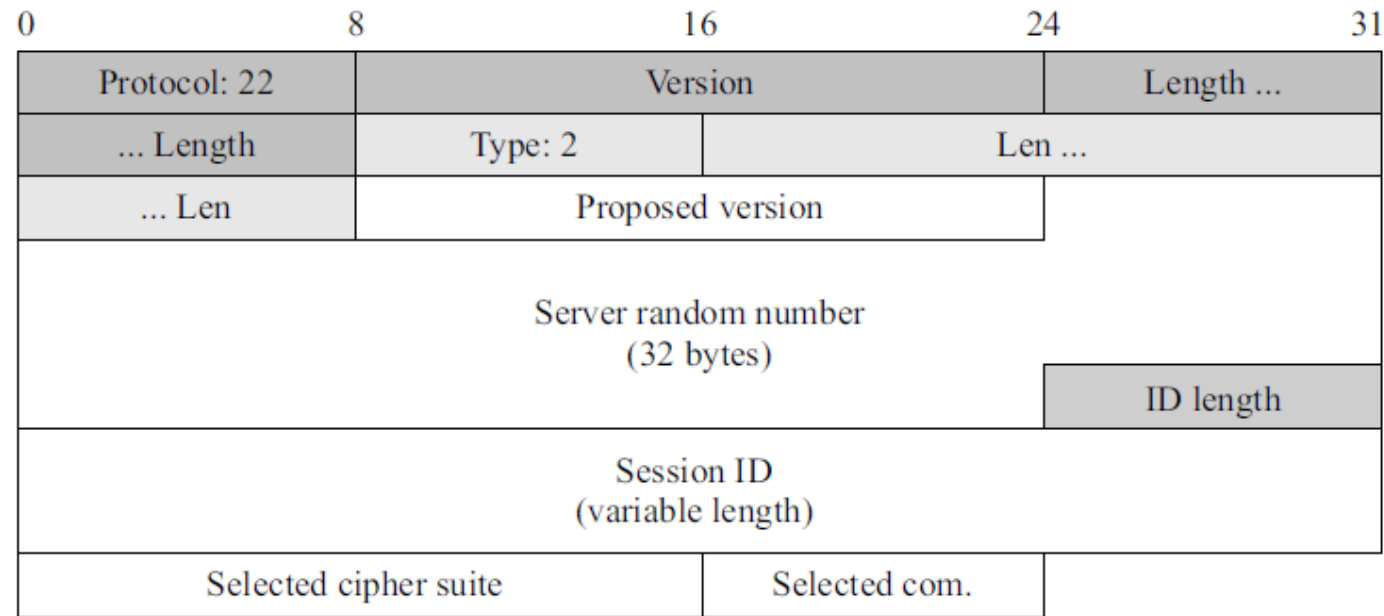
- **Version:** This 2-byte field shows the version of the SSL used. The version is 3.0 for SSL and 3.1 for TLS. Note that the version value, for example, 3.0, is stored in two bytes: 3 in the first byte and 0 in the second.
- **Client Random Number:** This 32-byte field is used by the client to send the client random number, which creates security parameters.
- **Session ID Length:** This 1-byte field defines the length of the session ID (next field). If there is no session ID, the value of this field is 0.
- **Session ID:** The value of this variable-length field is 0 when the client starts a new session. The session ID is initiated by the server. However, if a client wants to resume a previously stopped session, it can include the previously-defined session ID in this field. The protocol defines a maximum of 32 bytes for the session ID.
- **Cipher Suite Length:** This 2-byte field defines the length of the client-proposed cipher suite list (next field).
- **Cipher Suite List:** This variable-length field gives the list of cipher suites that the client supports. The field lists the cipher suites from the most preferred to the least preferred. Each cipher suite is encoded as a two-byte number.

ClientHello Message cntd..

- **Compression Methods Length:** This 1-byte field defines the length of client proposed compression methods.
- **Compression Method List:** This variable-length field gives the list of compression methods that the client supports. The field lists the methods from the most preferred to the least preferred. Each method is encoded as a one-byte number. So far, the only method is the NULL method (no compression). In this case, the value of the compression method length is 1 and the compression method list has only one element with the value of 0.

- The ServerHello message is the server response to the ClientHello message. The format is similar to the ClientHello message, but with fewer fields. Figure shows the format of the message.

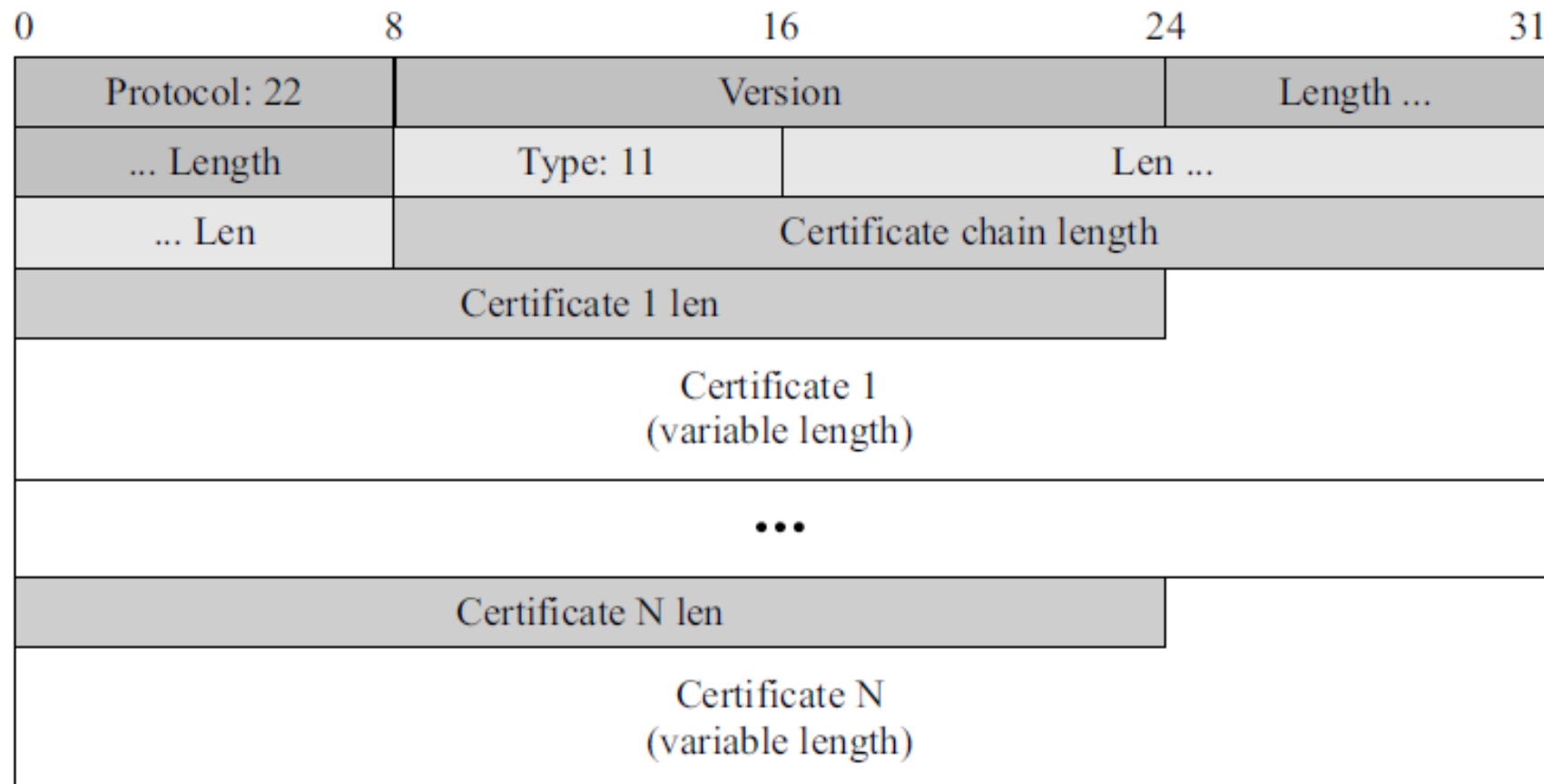
ServerHello Message



- The version field is same. The server random number field defines a value selected by the server. The session ID length and the session ID field are the same as those in the ClientHello message. However, the session ID is usually blank (and the length is usually set to 0) unless the server is resuming an old session. In other words, if the server allows a session to resume, it inserts a value in the session ID field to be used by the client (in the ClientHello message) if the client wishes to reopen an old session.
- The selected cipher suite field defines the single cipher suite selected by the server from the list sent by the client. The compression method field defines the method selected by the server from the list sent by the client.

Certificate Message

- The Certificate message can be sent by the client or the server to list the chain of publickey certificates. Figure shows the format.

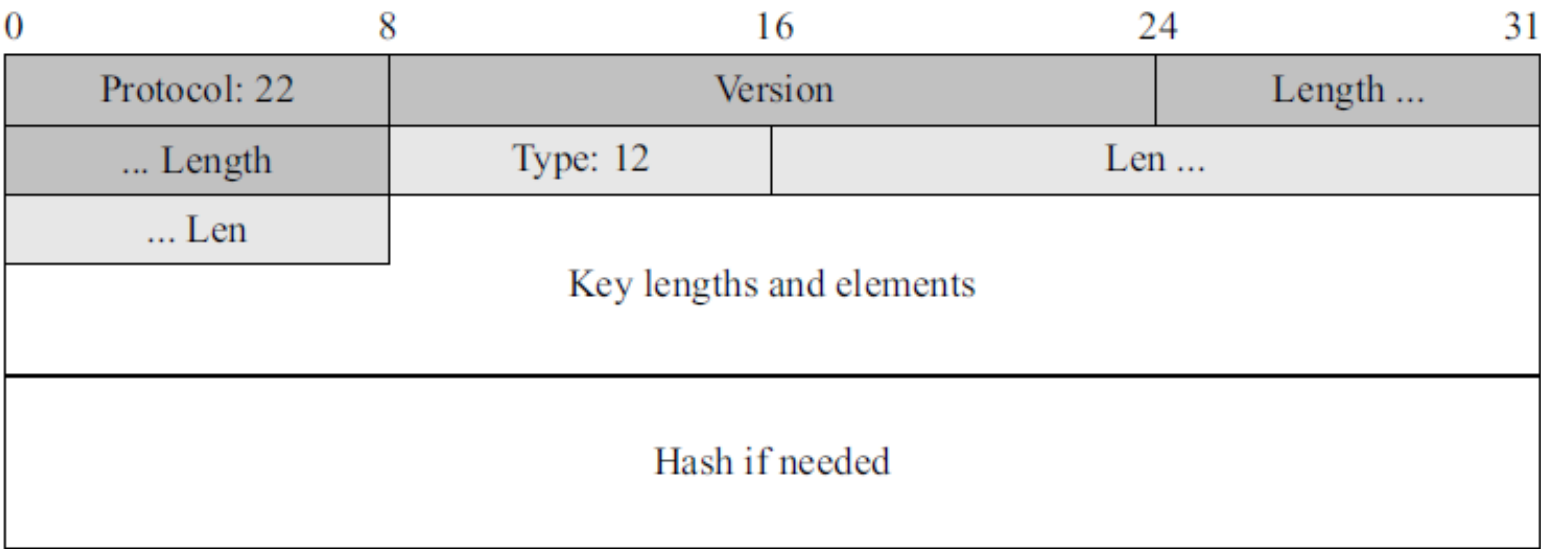


Certificate Message cntd..

- The Certificate message can be sent by the client or the server to list the chain of publickey certificates. Figure shows the format. The value of the type field is 11. The body of the message includes the following fields.
- **Certificate Chain Length.** This three-byte field shows the length of the certificate chain. This field is redundant because its value is always 3 less than the value of the length field.
- **Certificate Chain.** This variable-length field lists the chain of public-key certificates that the client or the server carries. For each certificate, there are two sub-fields: A three-byte length field, the variable size certificate itself.

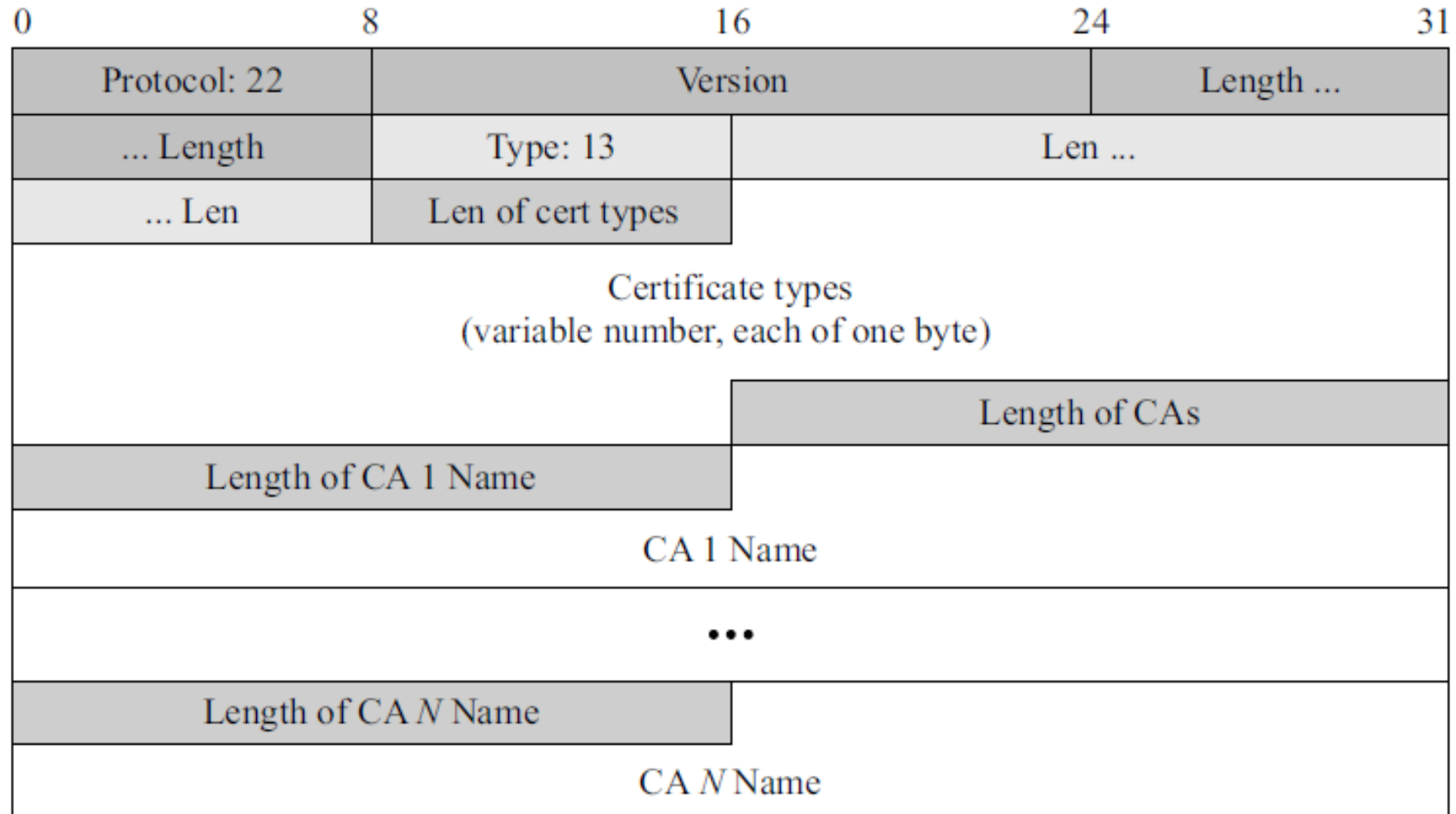
ServerKeyExchange message

- The **ServerKeyExchange** message is sent from the server to the client. Figure shows the general format.
- The message contains the keys generated by the server. The format of the message is dependent on the cipher suite selected in the previous message. The client that receives the message needs to interpret the message according to the previous information. If the server has sent a certificate message, then the message also contains a signed parameter.



CertificateRequest message

- The **CertificateRequest** message is sent from the server to the client. The message asks the client to authenticate itself to the server using one of the acceptable certificates and one of the certificate authorities named in the message. Figure shows the format.

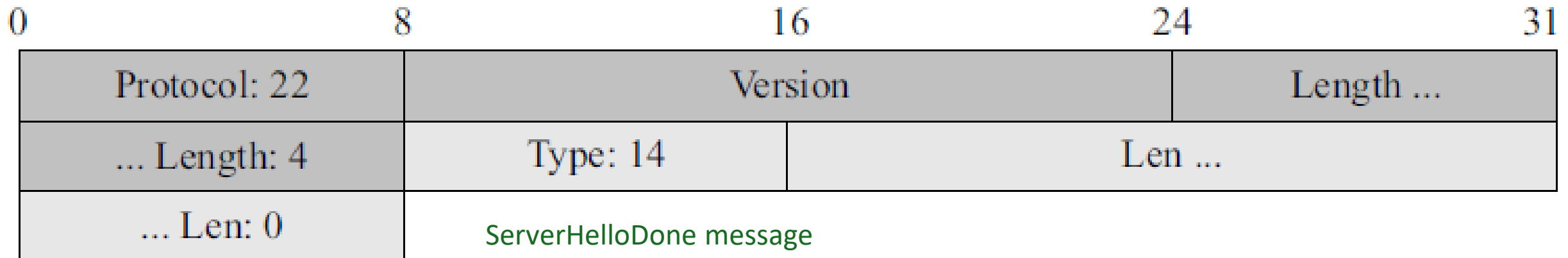


CertificateRequest message cntd..

- The value of the type field is 13. The body of the message includes the following fields:
 - **Len of Cert Types.** This one-byte field shows the length of the certificate types.
 - **Certificates Types.** This variable-length field gives the list of the public-key certificate types that the server accepts. Each type is one byte.
 - **Length of CAs.** This two-byte field gives the length of the certificate authorities (the rest of the packet)
 - **Length of CA x Name.** This two-byte field defines the length of the xth certificate authority name. The value of x can be between 1 to N.
 - **CA x Name.** This variable-length field defines the name of the xth certificate authority. The value of x can be between 1 to N.

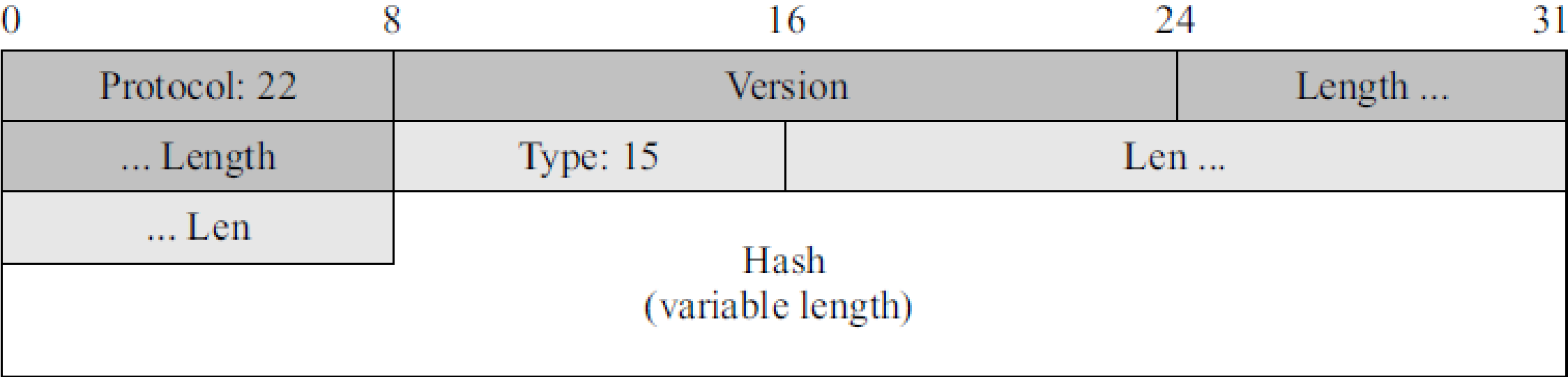
ServerHelloDone message

- The **ServerHelloDone** message is the last message sent in the second phase of handshaking. The message signals that phase II does not carry any extra information. Figure shows the format.



CertificateVerify message

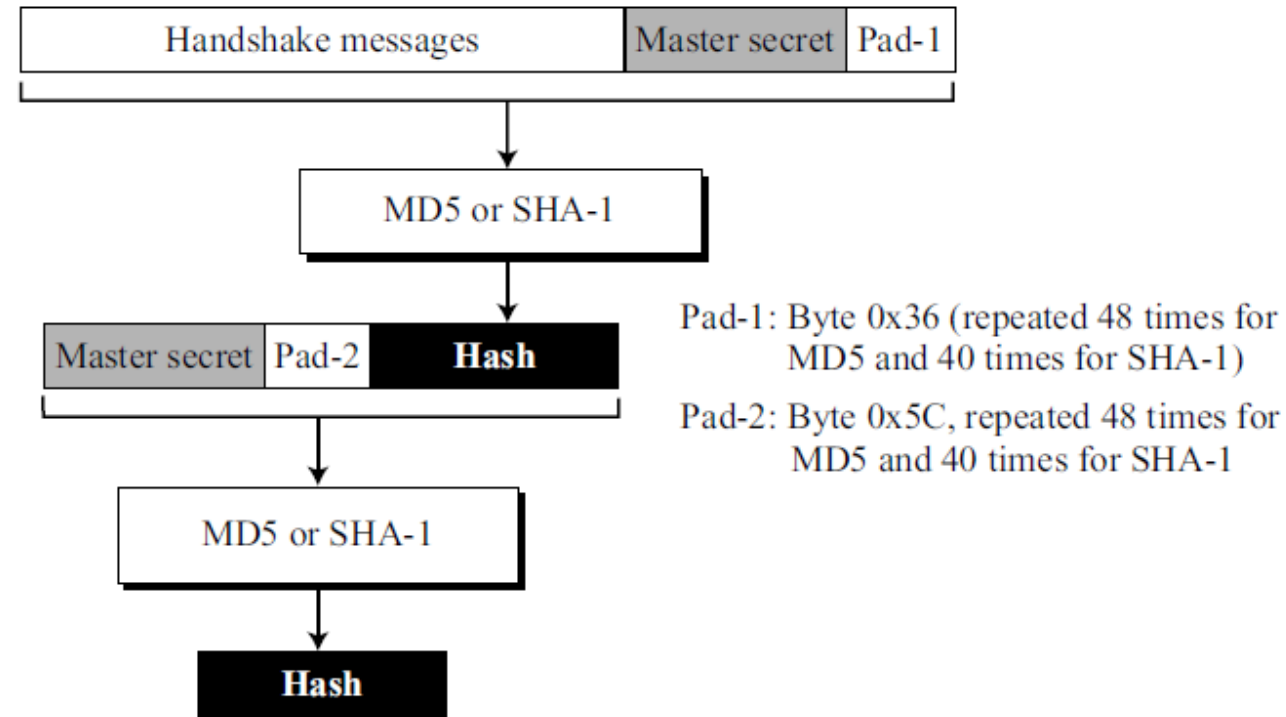
- The **CertificateVerify** message is the last message of Phase III. In this message, the client proves that it actually owns the private key related to its public-key certificate. To do so, the client creates a hash of all handshake messages sent before this message, and signs them using the MD5 or SHA-1 algorithm based on the certificate type of the client. Figure shows the format.



CertificateVerify message

CertificateVerify message cntd..

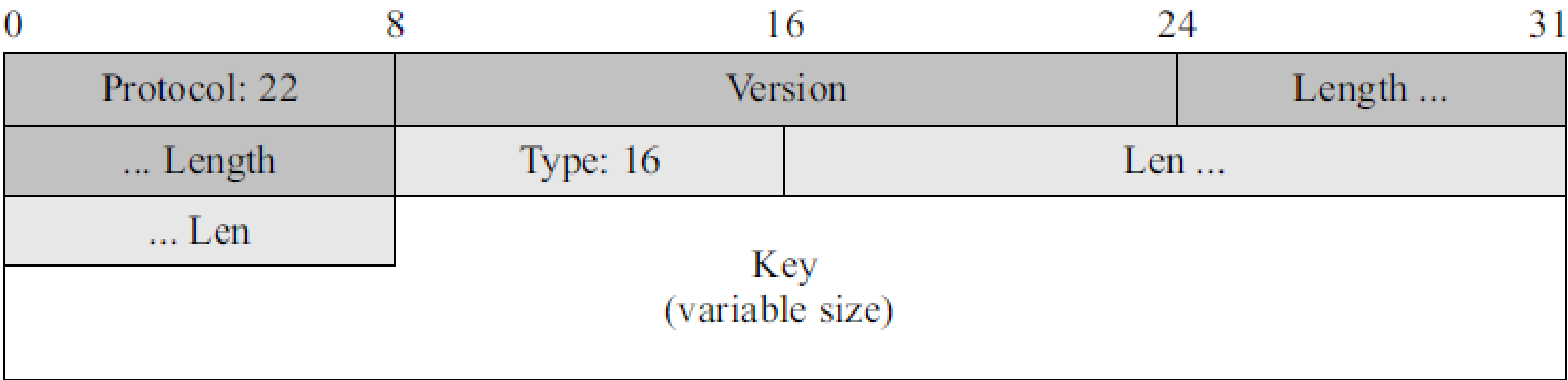
- If the client private key is related to a DSS certificate, then the hash is based only on the SHA-1 algorithm and the length of the hash is 20 bytes. If the client private key is related to an RSA certificate, then there are two hashes (concatenated), one based on MD5 and the other based on SHA-1. The total length is $16 + 20 = 36$ bytes.



Hash calculation for CertificateVerify message

Client key exchange message

- The ClientKeyExchange is the second message sent during the third phase of handshaking. In this message, the client provides the keys. The format of the message depends on the specific key exchange algorithms selected by two parties. Figure shows the general Idea.



ClientKeyExchange message

Finished message

- The Finished message shows that the negotiation is over. It contains all of the messages exchanged during handshaking, followed by the sender role, the master secret, and the padding. The exact format depends on the type of cipher suite used. The general format is shown in Figure. The below figure a shows that there is a concatenation of two hashes in the message. Figure b shows how each is calculated.

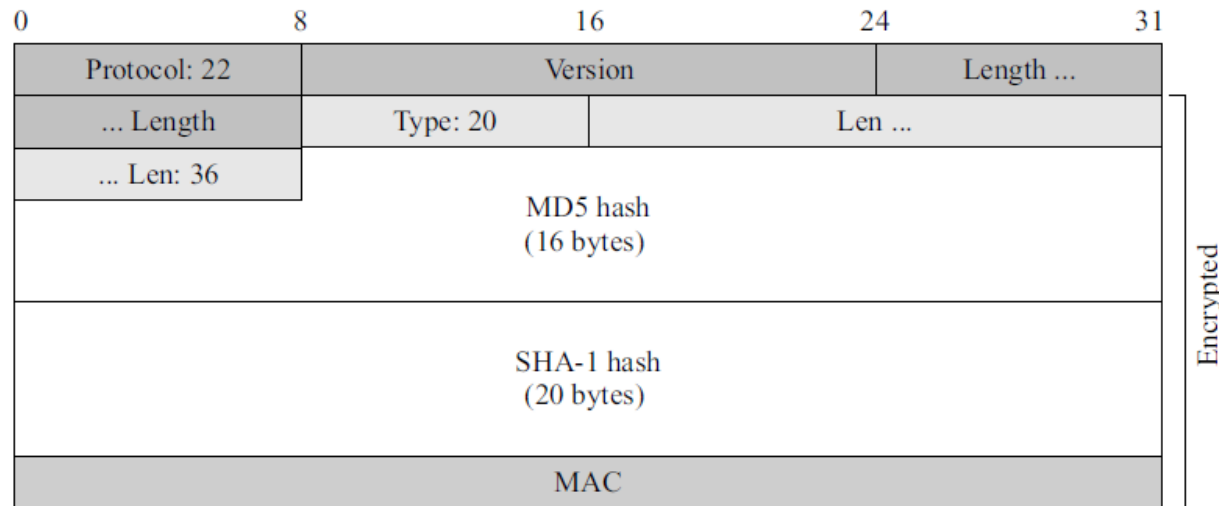


Figure a

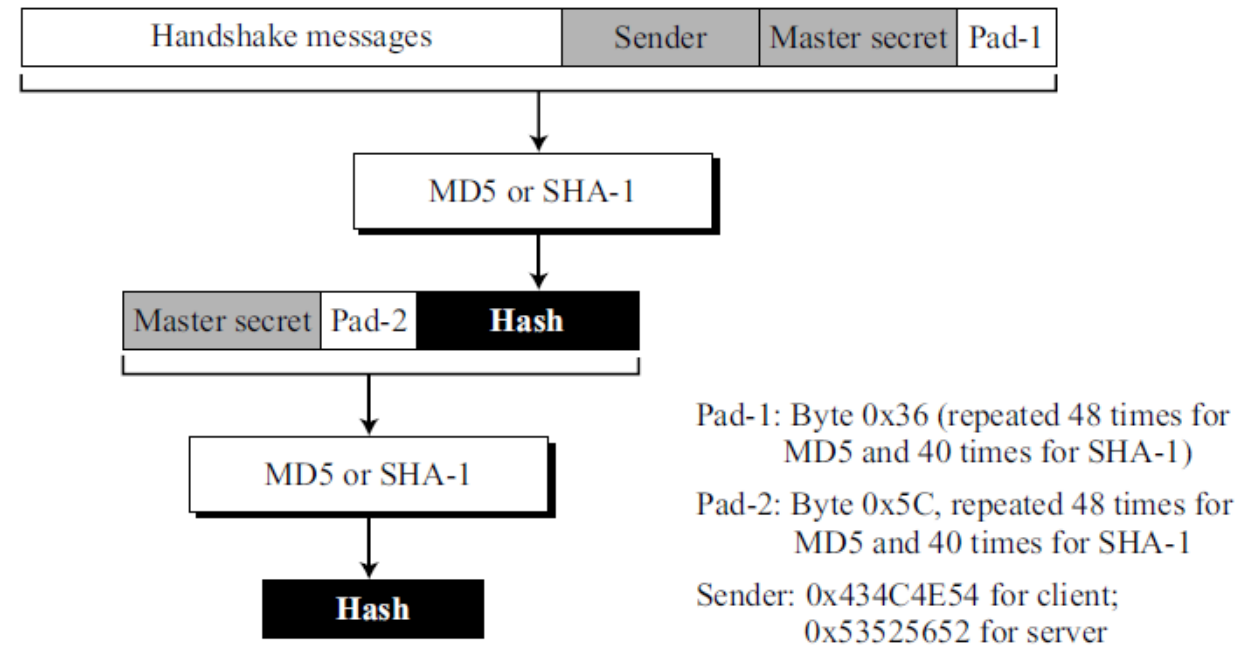


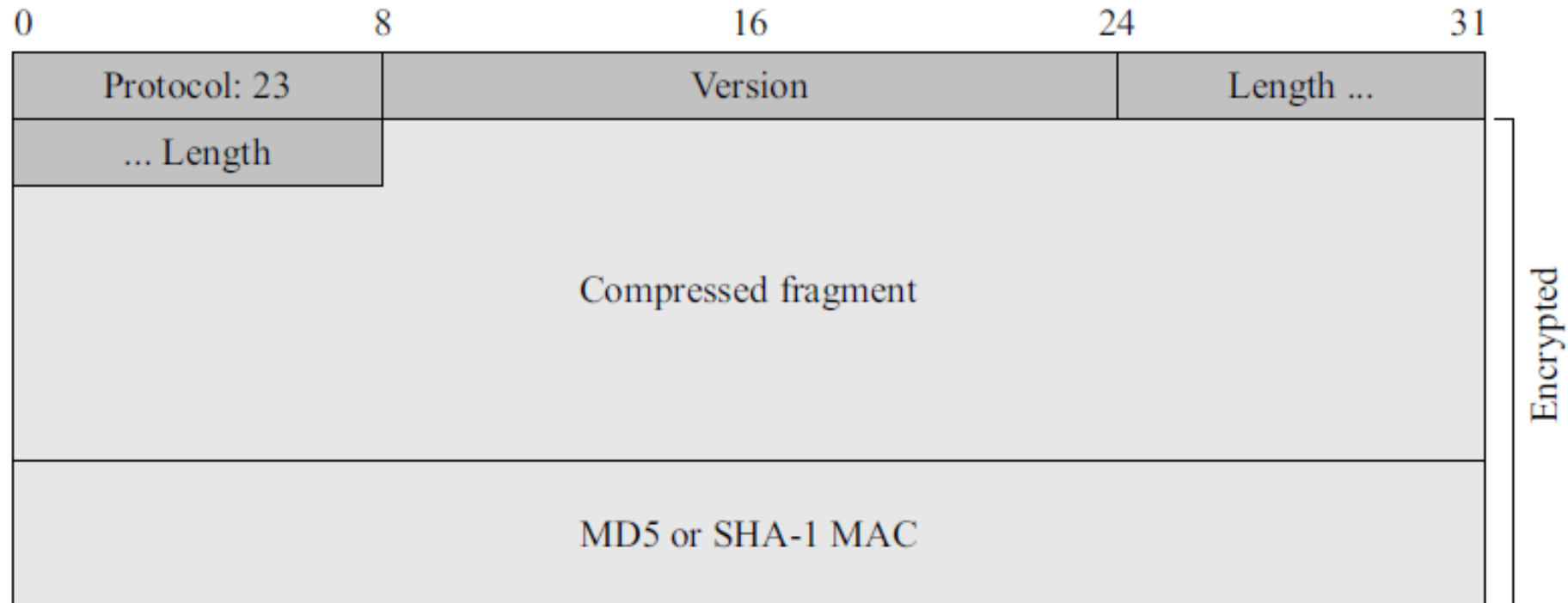
Figure b

Finished message cntd..

- When the client or server sends the Finished message, it has already sent the **ChangeCipherSpec** message. In other words, the write cryptographic secrets are in the active state. The client or the server can treat the Finished message like a data fragment coming from the application layer. The Finished message can be authenticated (using the MAC in the cipher suite) and encrypted (using the encryption algorithm in the cipher suite).

Application Data

- The Record Protocol adds a signature (MAC) at the end of the (possibly compressed) fragment coming from the application layer and then encrypts the fragment and the MAC. After adding the general header with protocol value 23, the Record message is transmitted. Note that the general header is not encrypted. Figure shows the format.



Record Protocol message for application data